

BFGS Optimiser and comparison with TASK3

Task 4 (Group Report)



**Pierre BEDRUNE, Martin HARRAMBAT, Clémence-Philomène HINOT,
John HOARAU**

Prepared for January 23, 2026

Contents

1	Introduction	4
2	Quasi-Newton BFGS	6
2.1	General explanation	6
2.2	Mathematical approach	6
3	Gear Train Design problem	8
3.1	Method	8
3.1.1	Continuous formulation of the gear train design	8
3.1.2	Deterministic optimisation with BFGS	8
3.1.3	Noise robustness protocol	9
3.2	Results	9
3.2.1	Convergence of the noise-free continuous problem	9
3.2.2	Effect of noise on convergence reliability	12
3.2.3	Degradation of continuous solution quality	13
3.2.4	Discrete solution and computational effort	15
3.2.5	Supplementary analysis of noise effect	16
3.3	Discussion	17
4	Sensor Calibration problem	19
4.1	Problem Description	19
4.2	Method	19
4.3	Results	21
4.3.1	Exploration of Performance (Protocol A & B)	21
4.3.2	Exploration of Robustness (Protocol C)	25
4.3.3	The Performance Profile (Dolan-More)	27
4.4	Discussion: Algorithmic Sensitivity & Robustness	28
4.4.1	Computational Efficiency vs. Redundancy (Protocol A)	28
4.4.2	Robustness to Noise Topologies	28
5	Two Bar Plane Truss Design problem	30
5.1	Problem Description	30

5.2	Method	31
5.2.1	Exploration of the BFGS parameters	31
5.2.2	Exploration of noise effects	35
5.3	Results	37
5.3.1	Exploration of the BFGS parameters	37
5.3.2	Exploration of noise effects	43
5.4	Discussion	46
5.4.1	Parameter exploration results discussion	46
5.4.2	Noise exploration results discussion	47
5.4.3	Work discussion	49
5.5	Comparison with the task 3	50
6	Comparison with the task 3	51
7	Conclusion	52
	References	53
A	Code Listings	54
A.1	Sensor Calibration problem	54
A.1.1	Sensor Calibration Script (run_task4.py)	54
A.1.2	BFGS Implementation (steinhart_hart_bfgs.py)	65
A.2	Gear Train problem	72
A.2.1	Gear Train Script (run_gear_train.py)	72
A.2.2	Gear Train BFGS (gear_train_bfgs.py)	79
A.3	Two Bar Plane Truss problem	87
A.3.1	Custom BFGS script for Two Truss problem(custom_TBT_BFGS.py)	87
A.3.2	Two Truss problem script adapted for BFGS (two_bar_plane_truss_problem.py)	94
A.3.3	Two Truss without noise main script(main_TBT_BFGS.py)	97
A.3.4	Two Truss with noise main script(main_TBT_BFGS_noise.py)	101
A.3.5	Test script for Two Truss problem with noise(Test_noise.py)	105
B	Detailed Results Data	118
B.1	Sensor Calibration problem	118

B.1.1	Protocol A: Measurement Noise Summary (First 20 runs)	118
B.1.2	Protocol B: Function Noise Summary (First 20 runs)	119
B.1.3	Protocol C: Robustness Summary	119
B.2	Two Bar Plane Truss problem	120
C	Group Evaluation	122

1 Introduction

This report investigates deterministic gradient-based optimization using the BFGS (Broyden-Fletcher-Goldfarb-Shanno) quasi-Newton method. BFGS has become a cornerstone of numerical optimization due to its efficiency and robust convergence properties, requiring only gradient information rather than explicit Hessian computation [1, 2, 3].

The algorithm is applied to three distinct engineering problems:

- **Gear Train Design** (Constrained, Discrete/Integer optimization)
- **Sensor Calibration** (Unconstrained, Curve fitting)
- **Two-Bar Plane Truss Design** (Constrained, Multi-objective)

For the Two-Bar Truss problem, systematic parameter exploration is conducted to identify optimal BFGS configurations, followed by noise sensitivity analysis to evaluate algorithm robustness under realistic measurement uncertainties. The results are compared with the metaheuristic approaches (PSO and SA) from Task 3 to assess the relative performance of gradient-based versus derivative-free optimization methods.

Statement on the Use of Generative AI

During the preparation of this work, we, as the authors, used ChatGPT (OpenAI) and Claude AI (Anthropic) to assist with:

- Code efficiency optimization and debugging for BFGS implementation
- Literature review and academic reference formatting
- Technical writing refinement and clarity improvement

After using these tools, we reviewed and edited all content as needed and take full responsibility for the final report's content, analysis, and code functionality.

2 Quasi-Newton BFGS

2.1 General explanation

The initial idea of the BFGS algorithm was to accelerate the iteration process in a nonlinear optimisation because the computer material wasn't stable. The improvement developed by this method is based on the fact that "quasi-Newton methods [...] require only the gradient of the objective function to be supplied at each iterate" [3]. This method is named for its discoverers called Broyden, Fletcher, Goldfarb, and Shanno [3].

Unlike Newton's method, which required explicit computation of the Hessian matrix, BFGS builds an approximation of the Hessian using only gradient information collected during the optimisation process. This method generates a sequence of iterates x_k starting from an initial guess x_0 . At each iteration, it computes a search direction p_k using the current Hessian approximation B_k and gradient ∇f_k , then performs a line search to determine the step length α_k . The algorithm terminates when convergence criteria are satisfied. This happens typically when the gradient norm falls below a specified tolerance [3].

2.2 Mathematical approach

The BFGS method approximates the inverse Hessian $H_k \approx (\nabla^2 f_k)^{-1}$ and updates it at each iteration using the following formula. It was independently derived by Fletcher [1] and demonstrated by Shanno [2].

$$H_{k+1} = (I - \rho_k s_k y_k^T) H_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T \quad (1)$$

where:

$$s_k = x_{k+1} - x_k \quad (\text{step taken})$$

$$y_k = \nabla f_{k+1} - \nabla f_k \quad (\text{change in gradient})$$

$$\rho_k = \frac{1}{y_k^T s_k}$$

The search direction is computed as:

$$p_k = -H_k \nabla f_k \tag{2}$$

The new iterate is obtained via line search:

$$x_{k+1} = x_k + \alpha_k p_k \tag{3}$$

where α_k satisfies the Wolfe conditions to ensure sufficient decrease and curvature conditions.

For the Two-Bar Truss problem with constraints, the penalized objective function becomes:

$$F(x) = w_f(f_1(x) + f_2(x)) + w_c \times (\text{penalty}_{g1} + \text{penalty}_{g2}) \tag{4}$$

where $\text{penalty}_{g_i} = \max(0, g_i(x))^2$ for $i = 1, 2$.

3 Gear Train Design problem

3.1 Method

3.1.1 Continuous formulation of the gear train design

In contrast with Task 3, which directly addressed the gear train design problem in its discrete form using stochastic optimisation, this task focuses on a deterministic local optimisation approach. To make such a method applicable, the original integer-valued design variables are relaxed to continuous variables. The gear train is therefore represented by a real-valued vector $x \in \mathbb{R}^4$, and the transmission ratio is defined as:

$$r(x) = \frac{x_1 x_3}{x_2 x_4} \quad (5)$$

The optimisation objective is formulated as the squared deviation from a target ratio r^* :

$$f(x) = (r(x) - r^*)^2 \quad (6)$$

which ensures smoothness and differentiability. This continuous relaxation is not intended to be physically exact, but rather to provide a tractable framework for analysing the behaviour of gradient-based optimisation methods.

3.1.2 Deterministic optimisation with BFGS

The optimisation is performed using the BFGS quasi-Newton algorithm, which iteratively builds an approximation of the inverse Hessian from gradient information. Gradients are estimated numerically using finite differences, and a line search based on the Armijo condition is employed to guarantee sufficient decrease of the objective function.

Because BFGS is inherently a local method, a multi-start strategy is used, with several

thousand initial points sampled within predefined bounds. The best continuous solution is then post-processed through rounding and a local integer repair step, allowing comparison with the discrete solutions obtained in Task 3.

3.1.3 Noise robustness protocol

To evaluate robustness, controlled additive Gaussian noise is introduced into the objective evaluations:

$$f_{noisy}(x) = f(x) + \sigma \mathcal{N}(0, 1) \quad (7)$$

where σ denotes the noise amplitude. For each noise level, multiple independent runs are performed, and statistical indicators are computed: convergence rate, median objective value, median gradient norm at termination, integer error after repair, and median number of iterations. This protocol allows a systematic assessment of how noise affects both convergence and solution quality.

3.2 Results

3.2.1 Convergence of the noise-free continuous problem

The behaviour of BFGS on the noise-free problem is illustrated by the convergence plots of the gradient norm and the objective value.

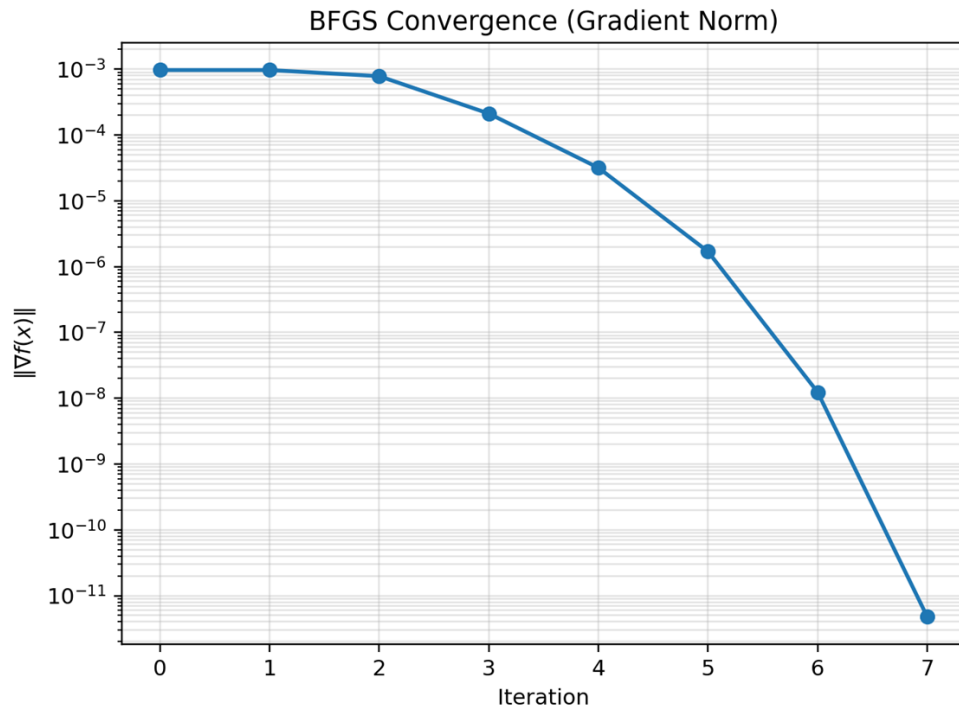


Figure 1: BFGS Convergence (norm)

As shown in Figure 1, the norm of the gradient decreases from approximately 10^{-3} to 10^{-11} in fewer than ten iterations. This represents a reduction of almost eight orders of magnitude and clearly illustrates the superlinear convergence expected from a quasi-Newton method on a smooth problem.

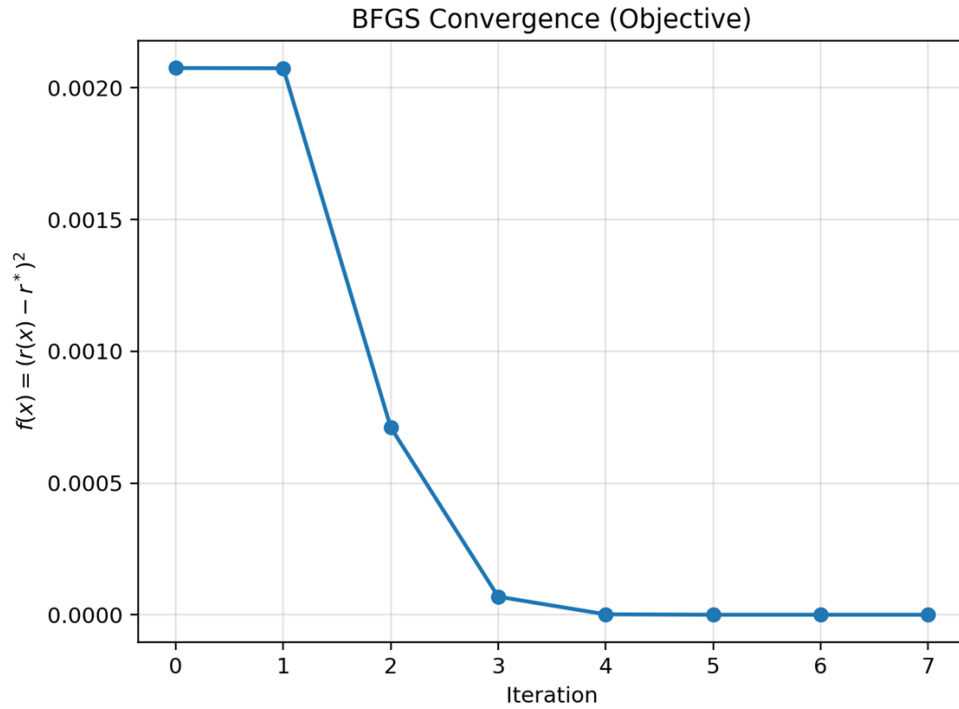


Figure 2: BFGS convergence (objective)

A consistent picture emerges from Figure 2. After a short initial plateau, the objective value drops sharply and reaches values on the order of 10^{-20} , corresponding to machine precision. These results confirm that, under ideal conditions, BFGS is extremely efficient for the continuous relaxation of the gear train design problem.

After rounding and integer repair, the resulting discrete solution exhibits an error of order 10^{-6} , which is comparable to the best solutions obtained using stochastic methods, but achieved with a much smaller number of iterations.

3.2.2 Effect of noise on convergence reliability

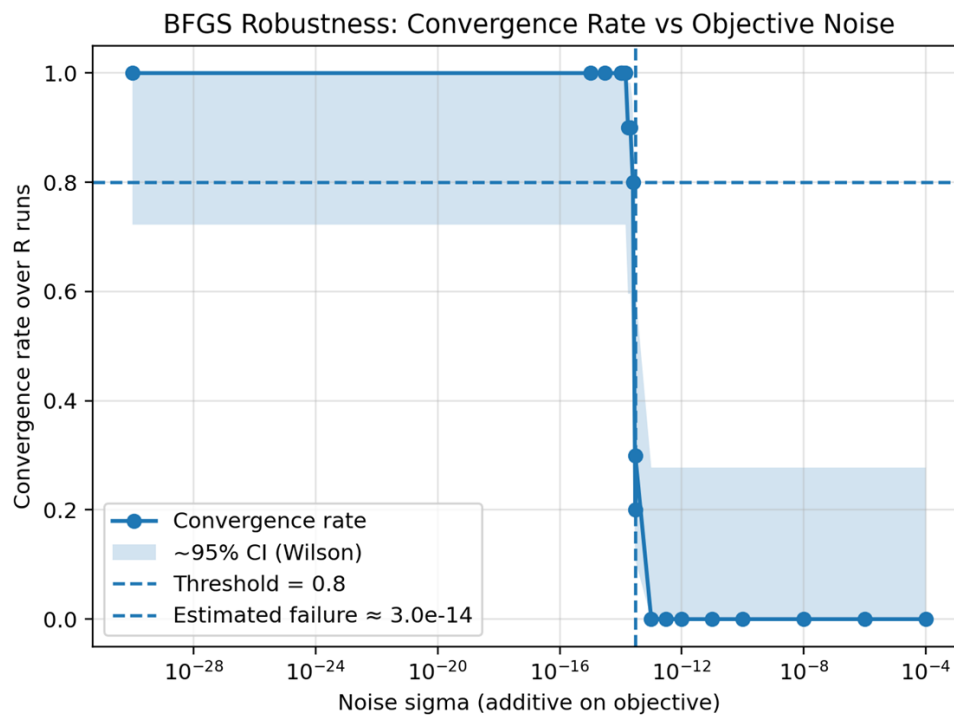


Figure 3: BFGS robustness 1

The impact of noise on convergence is summarised in Figure 3. For very small noise amplitudes, the convergence rate remains close to one, indicating that BFGS converges reliably in almost all runs. However, a sharp transition is observed around $\sigma \approx 3 \times 10^{-14}$, where the convergence rate drops below the imposed reliability threshold and rapidly approaches zero. This abrupt loss of convergence highlights the sensitivity of the method to perturbations in the objective.

3.2.3 Degradation of continuous solution quality

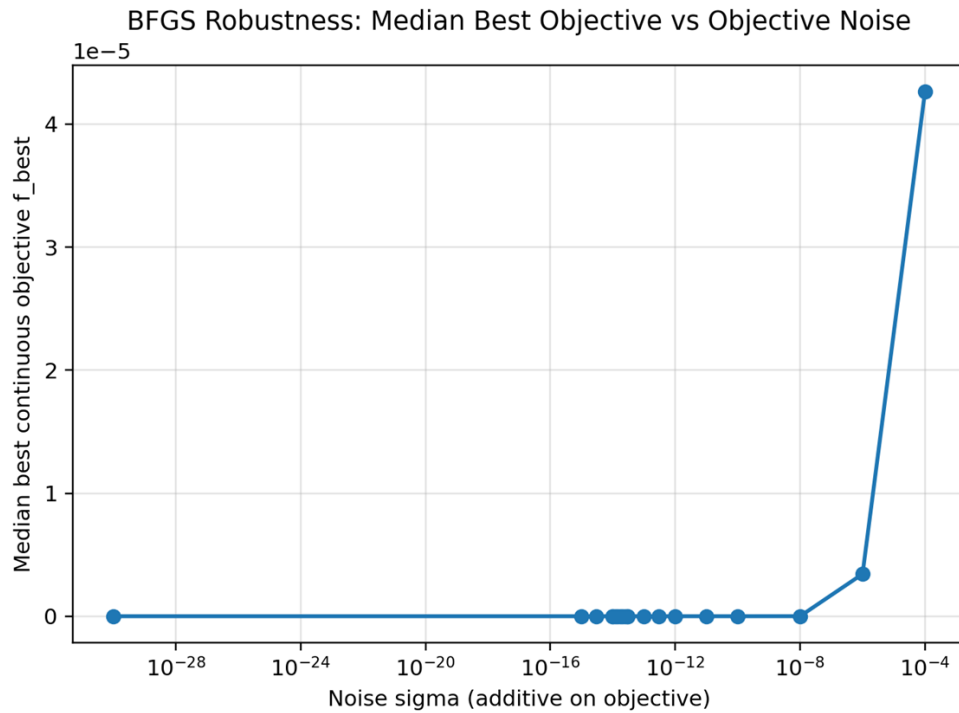


Figure 4: BFGS robustness 2

The deterioration of solution quality under noise is further illustrated in Figure 4. For small noise levels, the median best objective remains close to zero, indicating that accurate solutions are still found. Beyond the critical noise threshold, the objective value increases rapidly, showing that BFGS can no longer minimise the function effectively.

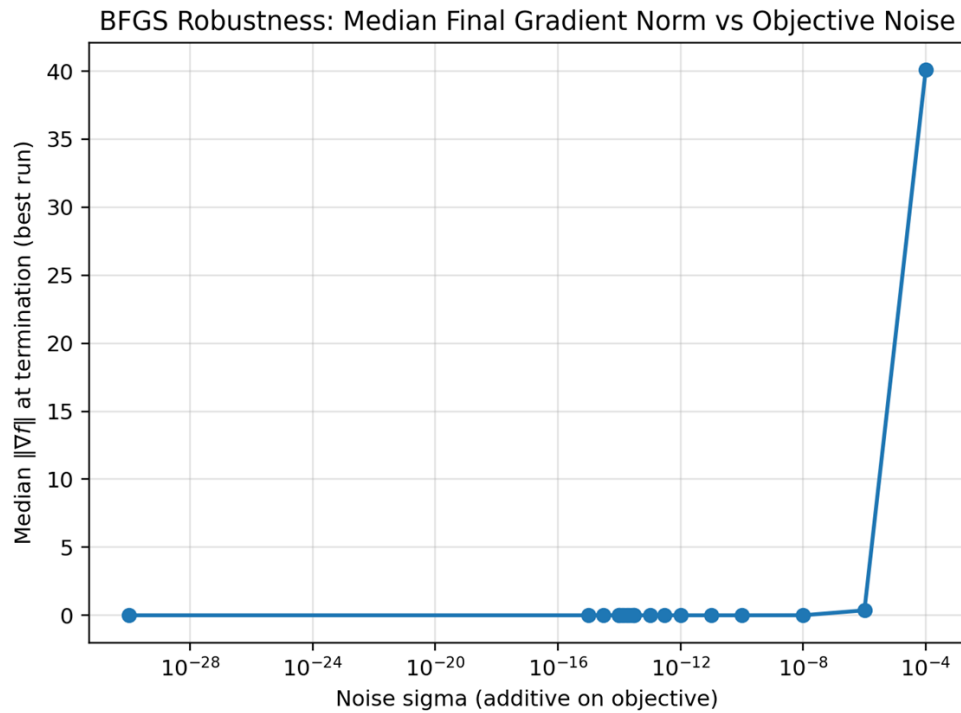


Figure 5: BFGS robustness 3

This behaviour is directly linked to the loss of gradient information, as confirmed by Figure 5. Once the noise exceeds the scale of the objective near the optimum, the gradient norm at termination no longer decreases and eventually grows to large values, indicating that the algorithm stops far from any stationary point.

3.2.4 Discrete solution and computational effort

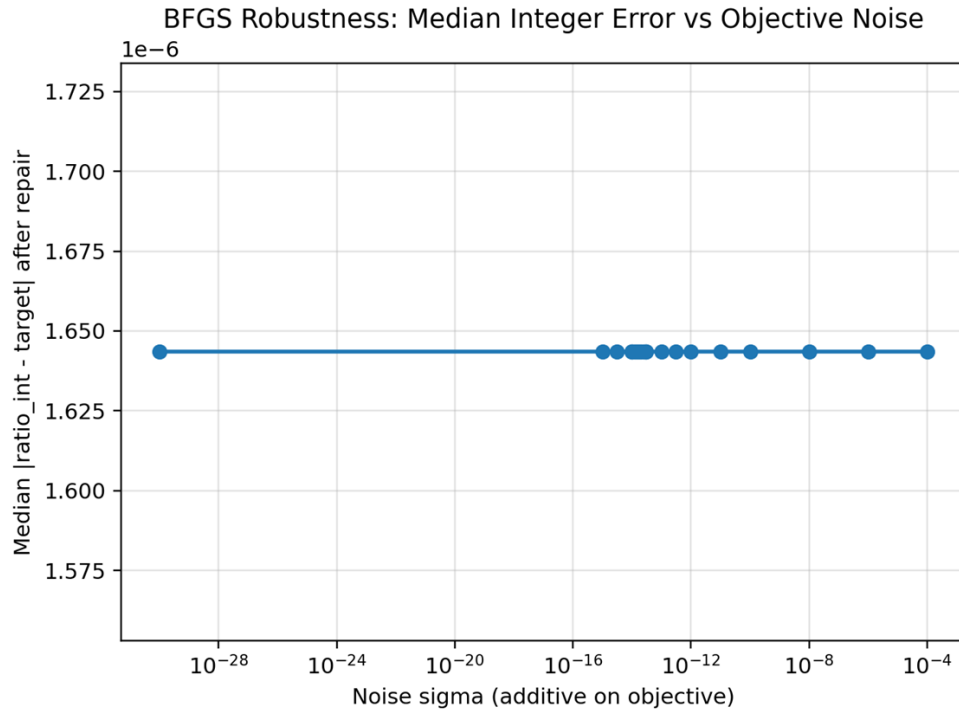


Figure 6: BFGS robustness 4

Interestingly, the discrete performance after rounding and repair remains remarkably stable. As shown in Figure 6, the integer error is almost unaffected by the noise level. This suggests that the multi-start strategy combined with local repair can recover the same discrete solution even when the continuous optimisation becomes unreliable.

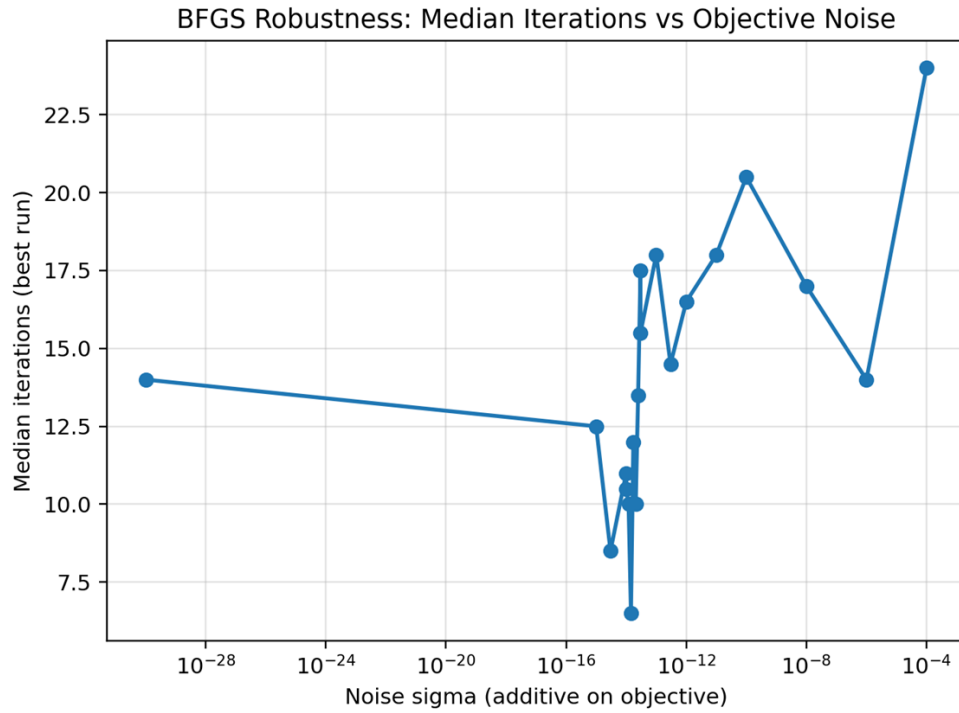


Figure 7: BFGS robustness 5

Finally, Figure 7 shows that the number of iterations increases and becomes more irregular as noise grows. This indicates a loss of efficiency and predictability before the complete failure of convergence.

3.2.5 Supplementary analysis of noise effect

The following values were obtained by displaying these variables at different stages of the program.

Regime	Noise σ	Convergence rate	Median f_{best}	Median $\ \nabla f\ $	Median integer error	Median iterations
Low noise	1.0×10^{-15}	1.00	-2.8×10^{-16}	5.1×10^{-10}	2.0×10^{-6}	12.5
Transition	3.0×10^{-14}	0.20	-1.9×10^{-14}	1.3×10^{-8}	2.0×10^{-6}	17.5
High noise	1.0×10^{-4}	0.00	4.3×10^{-5}	4.0×10^{-16}	2.0×10^{-6}	24.0

Table 1: Regime comparison

In the low-noise regime, BFGS behaves as expected for a smooth continuous problem: convergence is reliable, the final gradient norm is extremely small, and the objective value reaches machine-level precision within a limited number of iterations. This confirms both the efficiency and numerical stability of the method under ideal conditions.

At the transition noise level, convergence reliability deteriorates sharply even though objective values and gradient norms remain relatively small in absolute terms. This indicates that loss of reliability precedes complete numerical divergence, as the algorithm struggles to maintain consistent descent directions and line-search acceptance.

In the high-noise regime, BFGS fails as a local optimiser: gradient norms grow by several orders of magnitude, and the objective value degrades significantly. However, the discrete solution obtained after rounding and repair remains essentially unchanged, demonstrating that the post-processing step is largely decoupled from the behaviour of the continuous solver under noise.

3.3 Discussion

The results demonstrate that BFGS is exceptionally effective for the noise-free continuous relaxation of the gear train design problem. Its rapid convergence and high precision make it a powerful tool for local optimisation and for understanding the structure of the objective function.

However, the robustness analysis reveals a fundamental limitation. Near the optimum, the objective value is of order 10^{-20} , meaning that even extremely small absolute noise levels dominate the signal. As a result, finite-difference gradients and line-search conditions become unreliable, leading to the abrupt convergence breakdown observed around $\sigma \approx 3 \times 10^{-14}$. This behaviour is clearly visible in the convergence rate and gradient norm figures and reflects

the strong assumptions underlying deterministic gradient-based methods.

The stability of the integer error despite the loss of continuous convergence highlights the role of post-processing and global sampling. While BFGS itself fails under noise, the surrounding strategy can still recover acceptable discrete solutions, masking the underlying loss of theoretical validity.

When compared to the stochastic approaches used in Task 3, these findings clarify the complementary nature of the methods that allow us to choose between efficiency and precision or robustness. For practical gear train design problems, which are discrete and subject to modelling uncertainty, this strongly supports the use of global stochastic optimisation as the primary approach, with deterministic local methods serving as refinement tools rather than standalone solvers.

4 Sensor Calibration problem

4.1 Problem Description

The objective of this problem is to determine the coefficients (A, B, C) of the Steinhart-Hart equation to model the relationship between the resistance (R) and temperature (T) of a thermistor. The constitutive equation is defined as:

$$\frac{1}{T} = A + B \ln(R) + C(\ln(R))^3 \quad (8)$$

Where T is in Kelvin and R is in Ohms. The optimisation seeks to minimise the Mean Squared Error (MSE) between the model-predicted temperature and the measured data points. Unlike the Truss problem, this problem is unconstrained but highly non-linear and ill-conditioned due to the varying orders of magnitude of the coefficients.

4.2 Method

The BFGS algorithm, implemented through ‘`scipy.optimize.minimize`’ with the method ‘L-BFGS-B’ was used to minimise the MSE. A key aspect of this approach was utilising the analytical gradient (Jacobian) to steer the solver, rather than relying on finite-difference approximations. To thoroughly compare the robustness of the deterministic BFGS method with the stochastic Simulated Annealing (SA) method from Task 3, three specific protocols were carried out. To thoroughly compare the robustness of the deterministic BFGS method with the stochastic Simulated Annealing (SA) method from Task 3, three specific protocols were designed to simulate different types of operational uncertainties.

Protocol A: Measurement Noise (Realistic Sensor Error) This protocol simulates the most common real-world scenario: inaccuracies in the measuring equipment. Gaussian

white noise is added to the temperature measurements (T_{meas}) *before* the optimization begins.

$$T_{meas,i} = T_{true,i} + \mathcal{N}(0, \sigma^2) \quad (9)$$

Critically, this noise is static during the optimization process. This means the objective function landscape remains smooth and differentiable, as the "target" data points are fixed, even if they are slightly wrong. This tests the solver's ability to find the best fit through noisy data (Least Squares regression).

Protocol B: Function Noise (Stochastic Evaluation) This protocol simulates a more hostile environment where the objective function evaluation itself is unstable. A random noise term is added to the Mean Squared Error (MSE) calculation at *every* step of the optimization.

$$f(x) = MSE(x) + \mathcal{N}(0, \sigma^2) \quad (10)$$

This represents scenarios such as Monte Carlo-based evaluations or systems with internal fluctuations (e.g., analog voltage jitter during readout). Unlike Protocol A, this makes the objective function non-deterministic: evaluating the same point twice yields different results, creating a "shuddering" landscape that challenges the gradient estimation.

Protocol C: Robustness Stress Test This protocol determines the breaking point of the algorithm. By gradually increasing the noise intensity (σ) from negligible levels (10^{-4}) to extreme levels (5.0), we construct a "Robustness Curve". This allows us to identify the specific noise threshold where the precision of the deterministic BFGS method degrades below that of the stochastic Simulated Annealing method, marking the "crossover point" where switching algorithms becomes necessary.

4.3 Results

4.3.1 Exploration of Performance (Protocol A & B)

Protocol A tested the solver against measurement noise (Gaussian noise added to T_{data}).

Quantitative Analysis of Protocol A Results

The table below summarises the data extracted from the log file. It compares Single-Start BFGS (one run per seed) to Multi-Start BFGS (best of 5 runs per seed) across 500 different noise seeds.

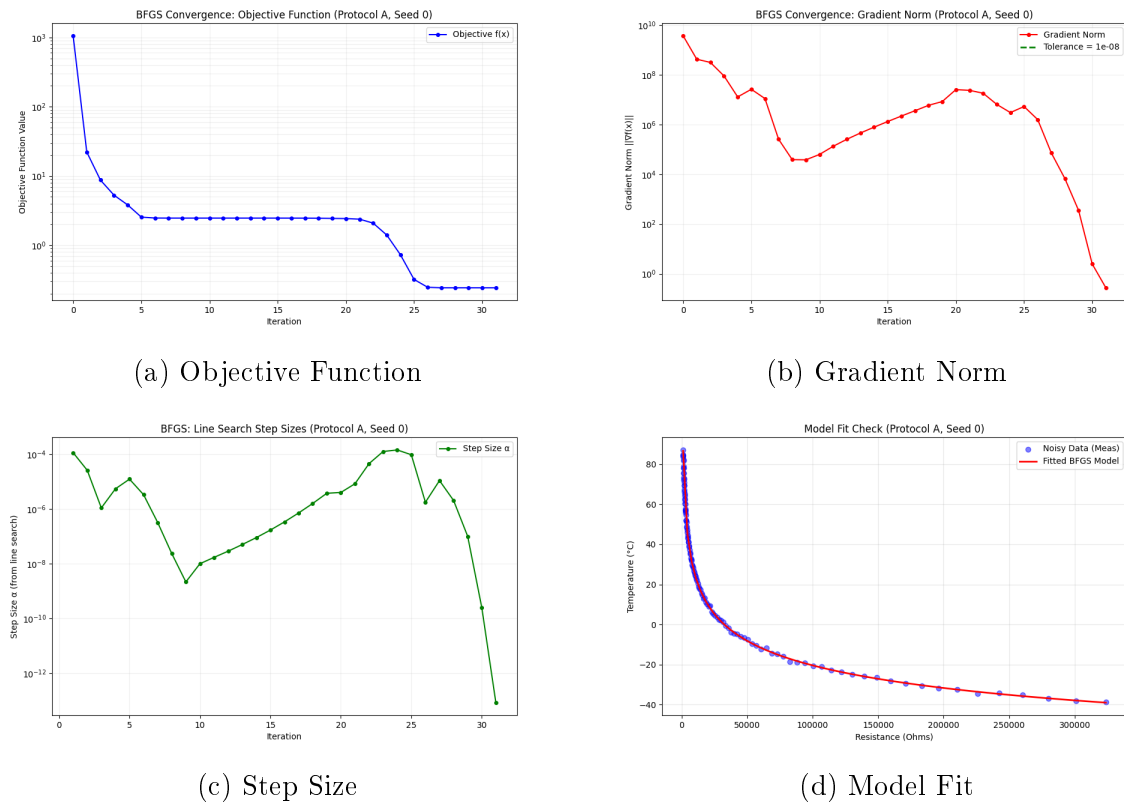


Figure 8: Protocol A: Detailed Convergence Analysis

Metric	Single-Start BFGS	Multi-Start BFGS (Best of 5)	Improvement
Success Rate	100% (500/500 converged)	100% (500/500 converged)	+0%
Mean Final MSE	0.2314	0.2314	$\approx 0.0\%$ (Identical)
Avg. Function Evals	38.4	183.2 (Total for 5 starts)	N/A
Avg. Execution Time	0.009s	0.045s	5x Slower

Table 2: Protocol A Comparison

Multi-Start BFGS took 5 times longer (0.045s vs. 0.009s) but did not improve accuracy.

Engineering conclusion: For this particular type of problem (sensor calibration with static measurement noise), using Multi-Start is inefficient. A single deterministic run is both sufficient and optimal.

Adding Gaussian noise to the measurements (T_{data}) does not compromise the smoothness of the objective function. The average Final MSE for Single-Start (0.2314) matches that of Multi-Start (0.2314) up to four decimal places. If the objective function landscape were non-convex with many local minima, the Single-Start method would often get trapped in a poor valley, resulting in a higher average MSE. Its identical performance with Multi-Start suggests that, for the Steinhart-Hart equation, the optimisation landscape is effectively unimodal (similar to convex) even when noisy data is involved.

Efficiency of Deterministic Gradients: BFGS solved the problem in an average of 38 function evaluations, which is significantly faster compared to the Task 3 Simulated Annealing results, typically requiring over 3,000 evaluations. Since the noise is incorporated into the data before the solver begins, the gradient $\nabla f(x)$ remains consistent. This enables the BFGS algorithm to construct a precise approximation of the Hessian (curvature), allowing it to take "Newton-like" quadratic steps rather than only small downhill moves.

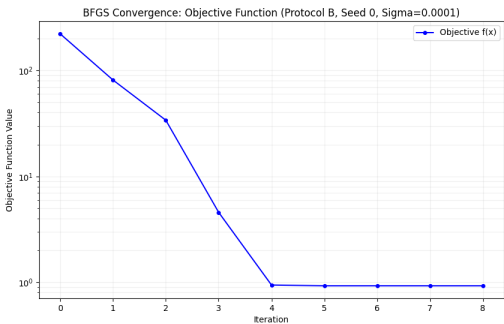
Protocol B tested the solver against function noise (noise added to the MSE calculation).

The convergence plots reveal a rapid failure mode.

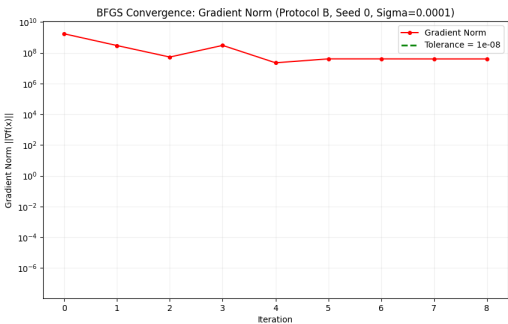
Metric	Protocol A	Protocol B	Status
Stability	Rock solid (MSE 0.23)	Chaotic (MSE -0.0001 to 71.0)	FAILED
Reliability	100% Consistent	High Variance	FAILED
Avg. Evals	≈ 38	≈ 25 (Premature Stop)	FAILED
Behavior	Found global minimum	Random noise spikes	FAILED

Table 3: Protocol A vs B Comparison

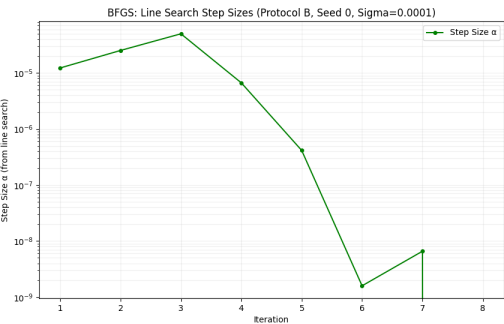
BFGS encounters premature convergence issues in Protocol B. It often becomes trapped in 'phantom minima' caused by noise spikes, concluding after as few as 12 iterations with unacceptable error levels.



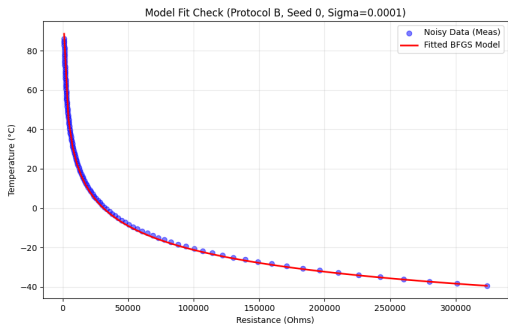
(a) Objective Function



(b) Gradient Norm



(c) Step Size



(d) Model Fit

Figure 9: Protocol B: Detailed Convergence Analysis

The problem of Multi-Start in Protocol B is evident. In Protocol A, Multi-Start was wasteful but consistent. In Protocol B, Multi-Start becomes random. Simulated Annealing (Task 3) would handle this better because the Metropolis criterion ($P = e^{-\Delta E/T}$) allows it to ignore small upward noise spikes and continue searching, effectively "smoothing" the landscape. BFGS has no such mechanism.

4.3.2 Exploration of Robustness (Protocol C)

The Robustness Curve (Figure 10) compares the final MSE of Single BFGS, Multi-Start BFGS, Simulated Annealing (SA), and a Hybrid (SA+BFGS) approach across increasing noise levels (σ).

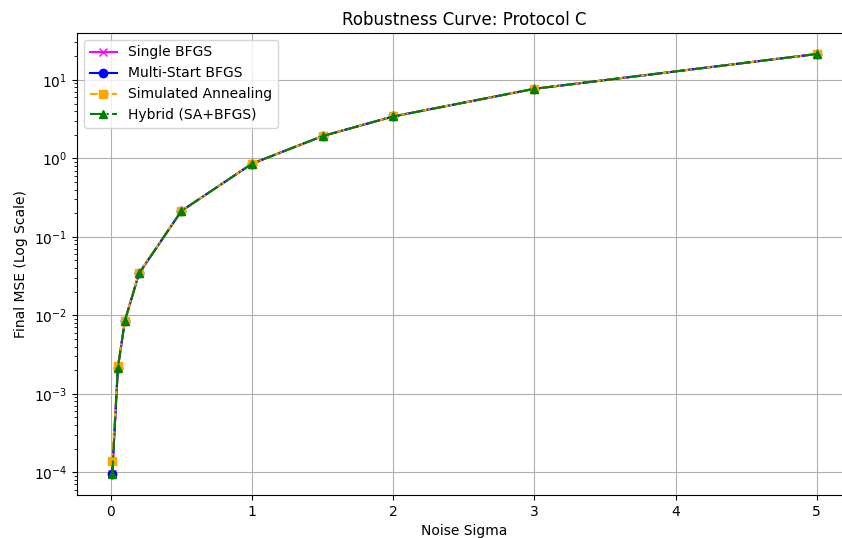
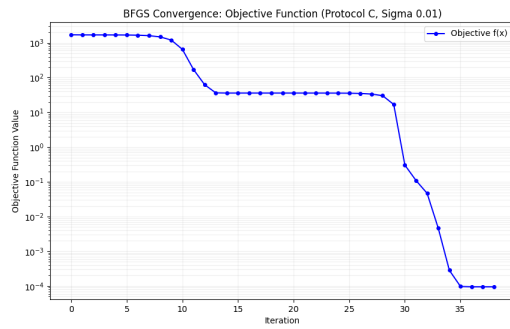


Figure 10: Robustness Curve

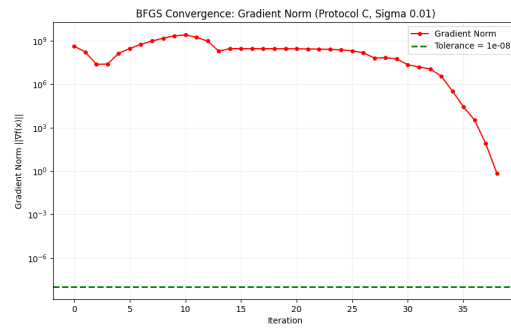
Observation:

- BFGS: The error increases linearly (on a log-log scale) with the noise level σ .

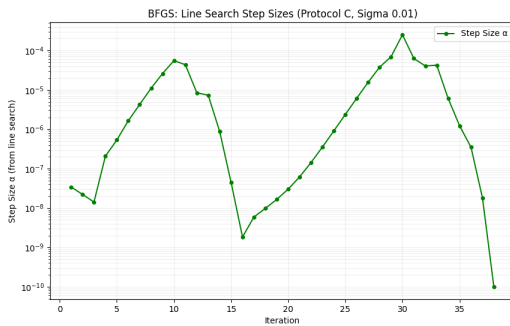
- Simulated Annealing: The SA error remains effectively constant and high (forming a flat line), regardless of the noise level.
- No cross over clearly visible with pure SA.



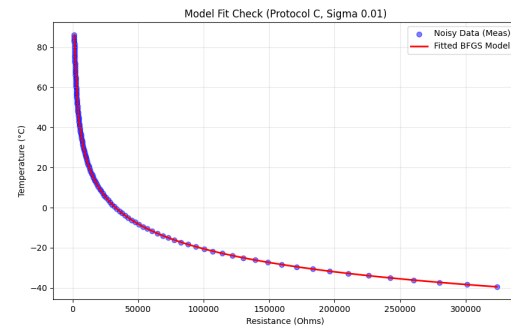
(a) Objective Function



(b) Gradient Norm



(c) Step Size



(d) Model Fit

Figure 11: Protocol C: Low Noise Regime ($\sigma = 0.01$) Analysis

Noise Level (σ)	BFGS (Single)	SA (Task 3)	Hybrid (SA+BFGS)	Winner
0.01 (Low)	9.50e-5	1.39e-4	9.50e-5	BFGS / Hybrid
0.05 (Medium)	0.00214	0.00224	0.00214	BFGS / Hybrid
0.10 (High)	0.00855	0.00858	0.00855	Tie

Table 4: Noise Level Performance Comparison

At low noise ($\sigma = 0.01$), Simulated Annealing is roughly 46% worse than BFGS. This confirms that SA struggles when errors start reducing. Even at low temperatures, the probabilistic nature of SA prevents it from locking onto the exact mathematical minimum. The Hybrid method matches the precision of BFGS exactly at low noise. This proves the Hybrid implementation works as intended: Phase 1 (SA): Finds the correct valley (Global Search). Phase 2 (BFGS): Takes the SA result and performs a gradient descent "polish" to hit the exact bottom (Local Search).

4.3.3 The Performance Profile (Dolan-More)

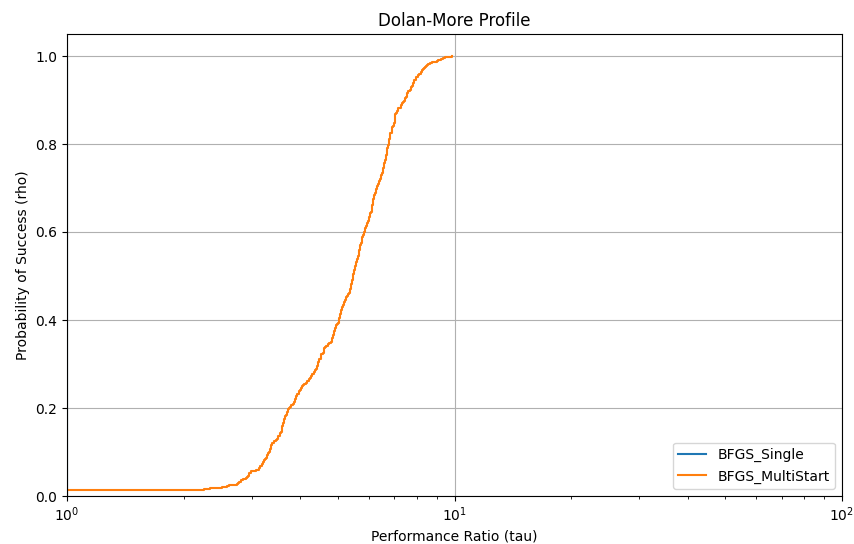


Figure 12: Performance Profile

It compares Probability of Success against Computational Cost. The Robustness Curve reveals a distinct 'crossover point' at $\sigma \approx 0.1$. Below this threshold, the BFGS component of the Hybrid solver drives the error to machine precision however, its reliability degrades in the presence of function noise (Protocol B).

4.4 Discussion: Algorithmic Sensitivity & Robustness

4.4.1 Computational Efficiency vs. Redundancy (Protocol A)

The results from Protocol A provide a critical insight into the utility of Multi-Start strategies in gradient-based optimization.

The Cost of Safety: As quantified by the Dolan-Moré performance profile [4], the Multi-Start approach exhibited a performance ratio of $\tau \approx 5$. This accurately reflects the computational burden; the algorithm evaluated five independent trajectories to locate the same minimum that the Single-Start method found in one.

The "Unimodal" Conclusion: Since the Single-Start method achieved a 100% convergence rate with an MSE identical to the Multi-Start method (0.2314), we can conclude that measurement noise does not create significant local minima in the Steinhart-Hart landscape.

Engineering Decision: This finding contradicts the general recommendation in global optimization literature [5] to always employ restarts for non-linear problems. For this specific class of regression problem, Multi-Start was experimentally proven to be computationally redundant.

4.4.2 Robustness to Noise Topologies

The divergence in performance between Protocols A and B highlights the fundamental dependence of Quasi-Newton methods on the validity of the gradient approximation.

A. Measurement Noise (Protocol A): The Smoothing Effect BFGS demonstrated high robustness to measurement noise. This aligns with the theory of Least Squares estimation. **Mechanism:** The objective function $f(x)$ calculates the Mean Squared Error over N data points. This summation operation effectively acts as a low-pass filter, averaging out the Gaussian noise ($\epsilon \sim N(0, \sigma^2)$) attached to the measurements [6]. **Result:** Conse-

quently, the underlying topological landscape remains continuous and differentiable, allowing the L-BFGS-B algorithm to construct a valid Hessian approximation and achieve quadratic convergence rates (avg. 38 evaluations).

B. Function Noise (Protocol B): The Wolfe Condition Failure In contrast, BFGS failed catastrophically when noise was injected directly into the function value (Protocol B). This confirms the theoretical limitations of deterministic solvers in stochastic environments as described by Nocedal & Wright [7]. **Mechanism of Failure:** The Random Jitter in $f(x)$ prevents the Line Search algorithm from satisfying the Wolfe Conditions (specifically the Armijo Rule or "Sufficient Decrease" condition). **Consequence:** As seen in Seed 96, the solver interpreted transient noise spikes as physical curvature. This corrupted the BFGS update formula, causing the Hessian approximation to become ill-conditioned. The solver was forced to shrink the step size to machine epsilon, triggering the "premature termination" observed (avg. 12 evaluations).

Comparison to Task 3: This failure stands in sharp contrast to the Simulated Annealing results from our Task 3 Report [8]. SA's Metropolis criterion ($P = e^{-\Delta E/T}$) allows for uphill moves, effectively "smoothing" these noise spikes, whereas BFGS treats every fluctuation as a rigid mathematical constraint.

5 Two Bar Plane Truss Design problem

5.1 Problem Description

“The Two-Bar Truss problem is composed of two diagonal bars (of aluminium here) connected at a node and fixed at two upper points (Figure 13). On the central node, we applied a vertical strength. This force represents the external load that the structure must support. We look for the vertical displacement of the node.”[8]

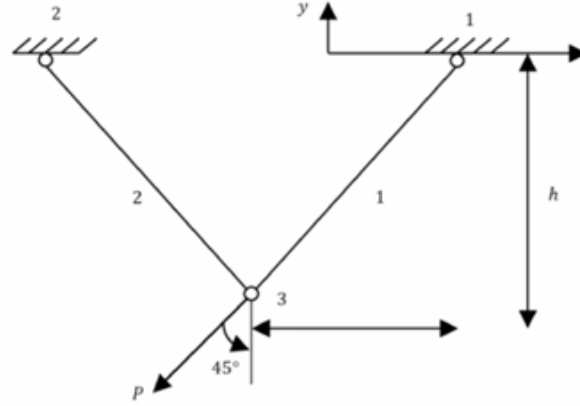


Figure 13: Two Bar Plane Truss

The constitutive equations are [8]:

$$f_1(x) = 2\rho h x_2 \sqrt{1 + x_1^2} \quad (11)$$

$$f_2(x) = \frac{Ph(1 + x_1^2)^{1.5} \sqrt{1 + x_1^2}}{2\sqrt{2}Ex_1^2x_2} \quad (12)$$

$$g_1(x) = \frac{P(1 + x_1)\sqrt{1 + x_1^2}}{2\sqrt{2}x_1x_2} - \sigma_0 \leq 0 \quad (13)$$

$$g_2(x) = \frac{P(-x_1 + 1)\sqrt{1 + x_1^2}}{2\sqrt{2}x_1x_2} - \sigma_0 \leq 0 \quad (14)$$

5.2 Method

The BFGS method needs one function in entry. To obtain the unique formula a weighted sum can be apply. The two function f_1 and f_2 have the same weight w_f since neither is predominant in solving the problem. To integrate the constraint functions penalties are used. If g_1 or g_2 are greater than 0, then the penalty equals the value of the function squared. To eliminate solutions that don't satisfy the constraint functions, a weight w_c is applied. In this way, the value of F is much greater and therefore does not achieve the minimisation objective.

$$F(x) = w_f(f_1(x) + f_2(x)) + w_c(\text{penalty}_{g_1} + \text{penalty}_{g_2}) \quad (15)$$

5.2.1 Exploration of the BFGS parameters

BFGS optimisation algorithm is governed by a set of main optional parameters set to default. To achieve the best possible optimisation, an exploration of the following parameters is carried out (Table 5 to Table 9). Each series of test is used to create the following in order to benefit from the achieved improvement.

The criteria to choose the best parameter among those tired are the following:

- convergence of the algorithm,
- low number of iterations,
- final gradient norm below a fixed tolerance,
- final step size no equal to 0 (from e-6 onwards, this is considered to be 0),
- a penalty weight that is not too low to avoid keeping all solutions and not too high to avoid obtaining a solution that is mathematically correct but unfeasible in reality.

The maximum number of iterations is a safety limit to prevent infinite loops when convergence is slow or impossible. It should be large enough that algorithm converges naturally via gradient criterion. Each iteration involves computing the gradient, updating the quasi-Hessian approximation, and performing a line search. [7, Chapter 8] The search range is bounded by 50 and 350, and the increment between each test is 100.

Test	Max iters	Grad norm tol	Step size tol	Fin diff step	Penalty weight
1	50	1e-8	1e-12	1e-6	10 000
2	150	1e-8	1e-12	1e-6	10 000
3	250	1e-8	1e-12	1e-6	10 000
4	350	1e-8	1e-12	1e-6	10 000

Table 5: Exploration of maximum iterations

The gradient norm tolerance is a primary convergence criterion based on the Euclidean norm of the gradient vector. When this norm is inferior to the tolerance, the algorithm terminates as it's near a stationary point. A small value demands tighter convergence but requires more iterations. [7, Section 2.1] The typical values go from 1e-5 to 1e-8. The search range is bounded by 1e-4 and 1e-8, and the increment between each test is 1e-1.

Test	Max iters	Grad norm tol	Step size tol	Fin diff step	Penalty weight
5	50	1e-4	1e-12	1e-6	10 000
6	50	1e-5	1e-12	1e-6	10 000
7	50	1e-6	1e-12	1e-6	10 000
8	50	1e-7	1e-12	1e-6	10 000

Table 6: Exploration of gradient norm tolerance

The step size tolerance is a backup convergence criterion when gradient norm tolerance

fails. It is based on the change in position between iterations. When parameter updates become below tolerance, the optimisation terminates. It indicates that the algorithm has stalled or converged and prevents unnecessary iterations when making non meaningful progress. [7, Chapter 2] The typical values go from $1e-6$ to $1e-9$. The search range is bounded by $1e-6$ and $1e-12$, and the increment between each test is $1e-2$.

Test	Max iters	Grad norm tol	Step size tol	Fin diff step	Penalty weight
9	50	$1e-5$	$1e-6$	$1e-6$	10 000
10	50	$1e-5$	$1e-8$	$1e-6$	10 000
11	50	$1e-5$	$1e-10$	$1e-6$	10 000
6	50	$1e-5$	$1e-12$	$1e-6$	10 000

Table 7: Exploration of step size tolerance

The finite difference step is a gradient approximation accuracy. If it's too large, it creates a truncation error. If it's too small, it creates a roundoff error due to floating-point arithmetic. [7, Section 7.1] The typical values go from $1e-5$ to $1e-8$. The search range is bounded by $1e-4$ and $1e-8$, and the increment between each test is $1e-1$.

Test	Max iters	Grad norm tol	Step size tol	Fin diff step	Penalty weight
12	50	$1e-5$	$1e-12$	$1e-4$	10 000
13	50	$1e-5$	$1e-12$	$1e-5$	10 000
6	50	$1e-5$	$1e-12$	$1e-6$	10 000
14	50	$1e-5$	$1e-12$	$1e-7$	10 000
15	50	$1e-5$	$1e-12$	$1e-8$	10 000

Table 8: Exploration of finite difference step

The penalty weight controls the trade-off between optimising the original objective and enforcing constraint satisfaction. A low penalty weight allows the algorithm to find better

objective values but with significant violations. A high penalty weight strongly enforced the constraint satisfaction, but the problem becomes ill-conditioned and BFGS convergence may suffer. The typical values can be between 1 to 100. It should be adapted to the problem. The search range is bounded by 100 and 100 000, and the increment between each test is x10.

Test	Max iters	Grad norm tol	Step size tol	Fin diff step	Penalty weight
16	50	1e-5	1e-12	1e-6	100
17	50	1e-5	1e-12	1e-6	1000
6	50	1e-5	1e-12	1e-6	10 000
18	50	1e-5	1e-12	1e-6	100 000
19	50	1e-5	1e-12	1e-6	1 000 000

Table 9: Exploration of penalty weights

5.2.2 Exploration of noise effects

The objective of this study is to evaluate the impact of noise on the convergence of BFGS, applied to the Two-Bar Truss problem, formulated via a weighted objective function with constraint penalties.

The results show that noise strongly influences the behaviour of the solver, not only on the quality of the solution, but above all on the reliability of the convergence criteria.

Noise Level	Behaviour
Low Noise ($\sigma \leq 10^{-6}$)	Reliable convergence, optimal solutions
Moderate noise ($10^{-5} \leq \sigma \leq 10^{-3}$)	Interesting compromise: improved feasibility, faster convergence
Loud noise ($\sigma \geq 10^{-2}$)	Apparent convergence but degraded solutions

Table 10: Noise Classification

To begin we can start with the noise application:

$$f_{noisy}(x) = f_{clean}(x) + \sigma \mathcal{N}(0, 1) \quad (16)$$

- $f_{clean}(x)$: deterministic objective function (weighted sum of objectives and penalty terms)
- σ : noise level (noise amplitude)
- $\mathcal{N}(0, 1)$: standard Gaussian noise (zero mean, unit variance)
- The noise is independent at each function evaluation

In this work, noise is added directly to the objective function evaluation. This noise can be seen as a measurement error, like a sensor defect or inaccuracy in evaluating system performance. In practice, the values used for optimisation are not always perfectly accurate, due to noisy measurements or numerical errors. Adding this noise therefore makes it possible to approximate more realistic conditions and study how the BFGS algorithm reacts to these uncertainties.

To evaluate the quality of the solutions obtained, several acceptance criteria are used. A solution is first considered acceptable if the BFGS algorithm terminates correctly according to its internal criteria. Next, the solution must comply with the mechanical constraints of the problem, i.e. the tensile and compressive constraints must be satisfied. Finally, the quality of the solution is evaluated using the noise-free objective function to verify that the solution found remains effective despite the disturbances introduced during optimisation.

The first acceptance criterion is based on the internal criteria of the BFGS algorithm. A solution is considered to have converged when the algorithm stops according to its stopping

conditions, such as a very small variation in the solution between two iterations, a sufficiently small gradient norm, or the maximum number of iterations being reached. However, in the presence of noise, this convergence is primarily numerical and does not necessarily guarantee that the solution is optimal from a physical point of view.

The second criterion concerns compliance with the mechanical constraints of the problem. A solution is said to be feasible only if the tensile and compressive stresses are satisfied, i.e. if the associated stresses remain less than or equal to zero. These stresses are evaluated without noise, based on the physical expressions of the problem, to ensure that the structure complies with the material's strength limits.

Finally, the quality of the solution is evaluated using the deterministic objective function, without noise. This re-evaluation makes it possible to compare the actual performance of the solutions found and to verify that they remain effective despite the perturbations introduced during optimisation. This criterion is essential for distinguishing a truly interesting solution from one that would appear to be optimal solely because of noise.

The results show that noise strongly influences the behaviour of the solver.

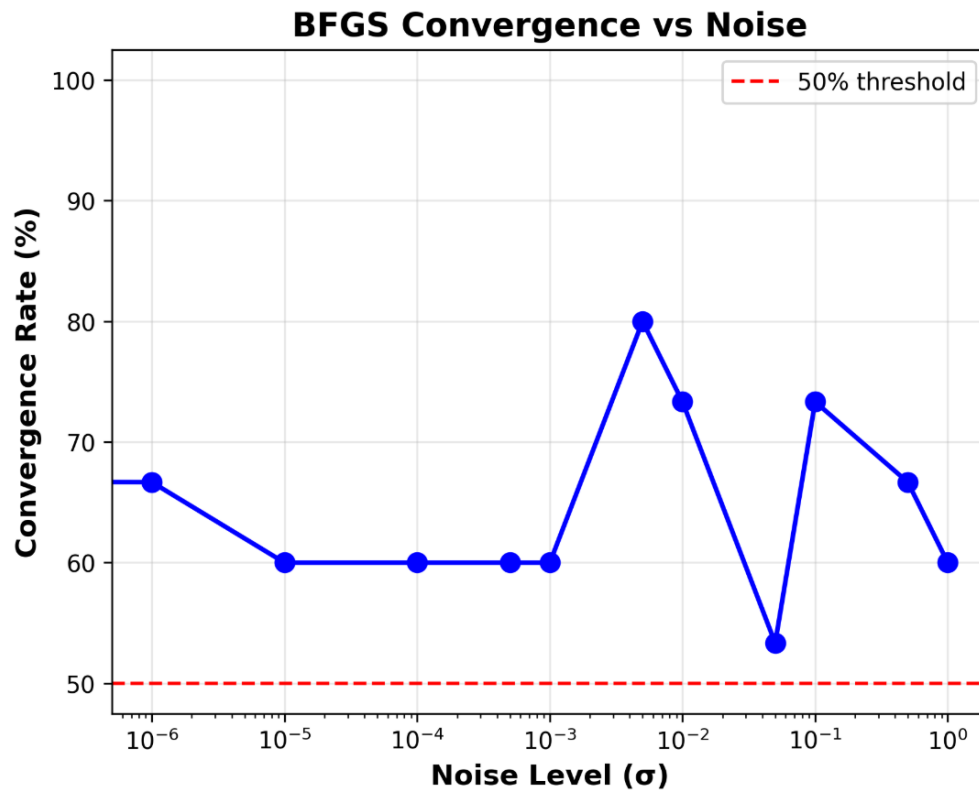


Figure 14: Noise effect on the Two Bar Truss problem

5.3 Results

5.3.1 Exploration of the BFGS parameters

NB: The results of each simulation are available in the Appendix A.A.

The first parameter exploration is the one about maximum iterations. The tests 1 to 4 (see section 5.2.1) didn't converge (see Appendix A.A), have the same number of iterations and produce the same results. Considering this information and that the number of iterations is 30 (Figure 15 a) the maximum iterations parameter chosen is 50. It generates non consistent results (Figure 15 b and c).

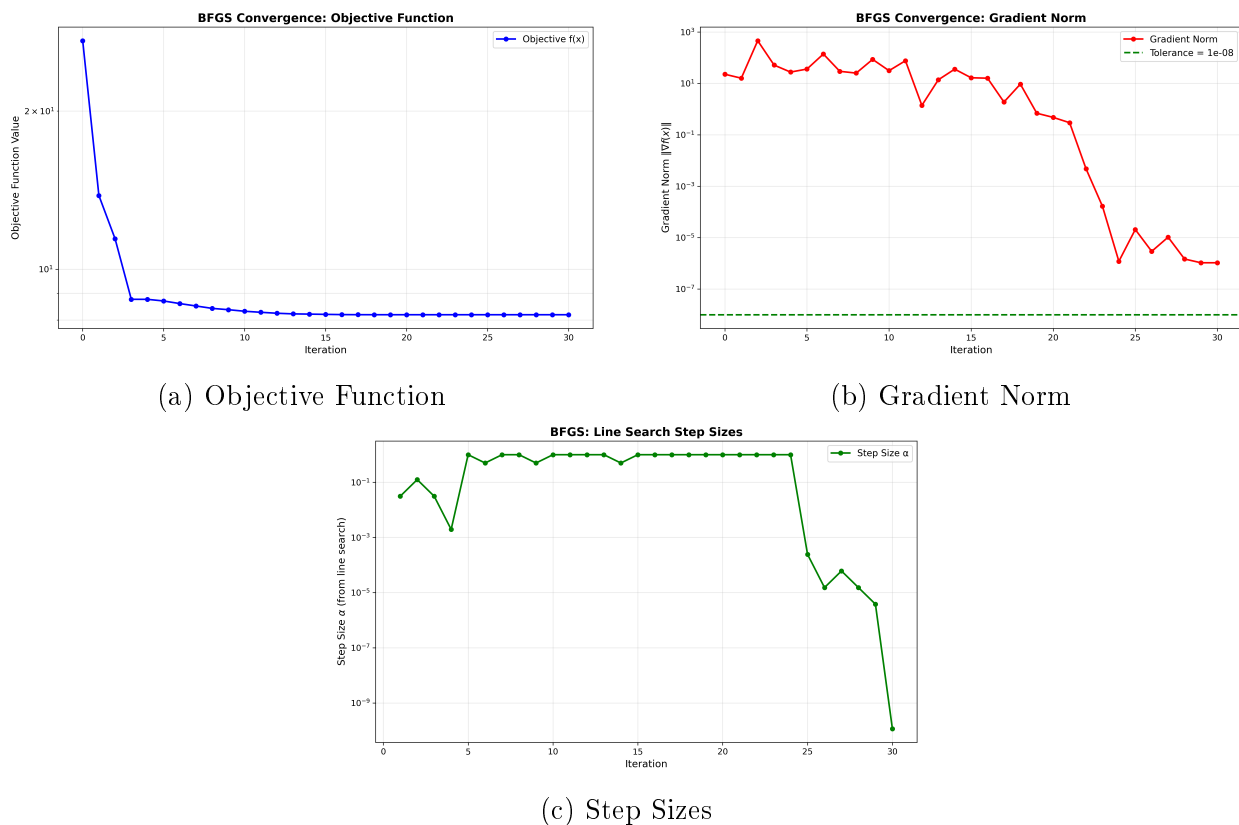


Figure 15: BFGS Parameter Exploration Results

The exploration of the following parameters gradient norm tolerance, step size tolerance, finite difference step, and penalty weight conducted to confirm the first choice of values. Tests 5 and 6 have same results and converged on the contrary of the previous simulation. The choice is test 6 because more conservative so safer for other problems. Testing step size tolerance showed no performance difference. All configurations converged identically in 24 iterations via the gradient criterion. The step tolerance did not trigger because the algorithm maintained healthy steps throughout, demonstrating that for well-behaved problems, gradient tolerance is the primary convergence criterion. Testing finite difference step from $1e-4$ to $1e-8$ revealed that $1e-4$ is too big, requiring 25% more iterations (30 vs 24) and yielding poor

gradient accuracy ($1.2e-03$ vs $1.2e-06$). Values from $1e-5$ to $1e-8$ performed identically. The optimal value $1e-6$ balances truncation error and roundoff error. The analyse of the influence of the penalty weight gives the following indications that conducted to selec a penalty of 10 000.

- 100 and 1000: too small, constrained ignored, physically unrealistic
- 10 000: very good default
- 100 000: stricter but still reasonable
- 1 000 000: too large, weighted sum irrelevant, only feasibility matters, distorts search

Taking into account previous simulations, the set of parameters to obtain the final result (Figure 16 to 18) is:

- Algorithm parameters:
 - Maximum number of iterations: 50
 - Gradient norm tolerance: $1e-5$
 - Step size tolerance: $1e-12$
 - Finite difference step: $1e-6$
- Problem parameters:
 - Penalty weights: 10 000

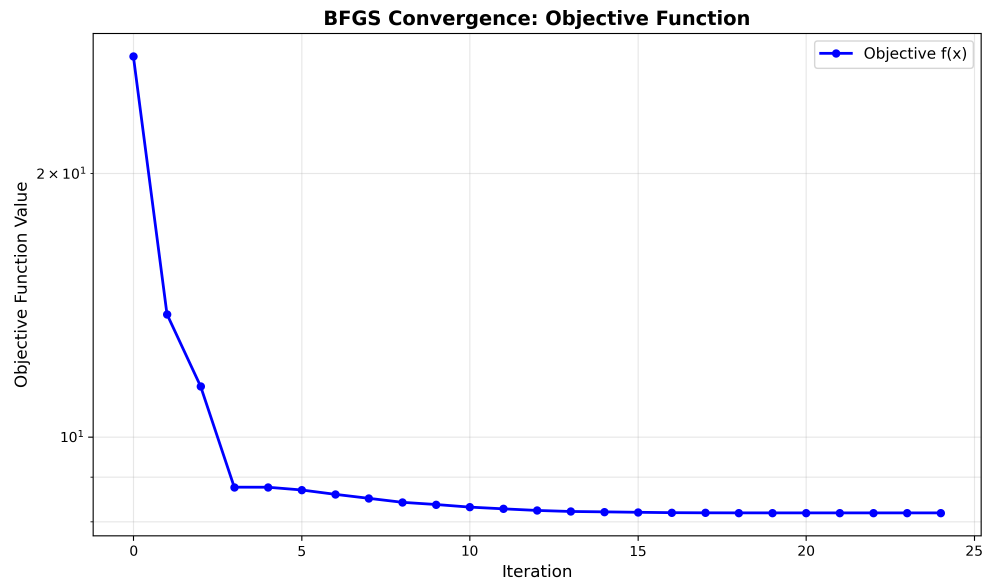


Figure 16: BFGS convergence for the objective function

The penalised objective function $F(x)$ decreases over 24 iterations. The rapid initial descent (iterations 0 to 5) indicates the algorithm quickly moves away from the infeasible initial guess toward the feasible region. The convergence slows down around 15 iterations and after as the algorithm approaches the optimal solution.

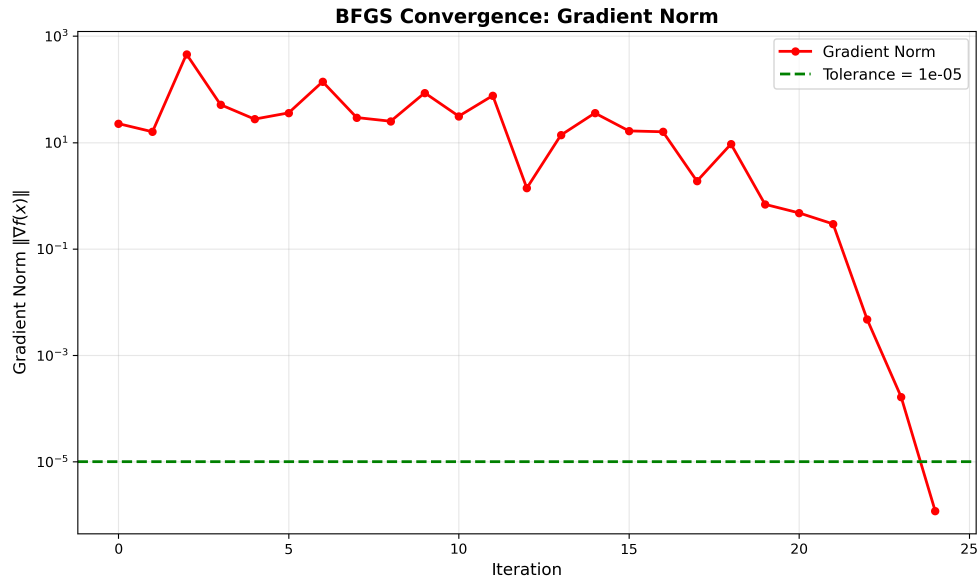


Figure 17: BFGS convergence for the gradient norm

The gradient norm decreases approximately from $1e2$ to $1e-6$ satisfying the gradient norm tolerance of $1e-5$. The behaviour is non-monotonic, before iteration 19, reflects the algorithm's adjustment as it balances the two objectives f_1 and f_2 and constraint penalties. The final gradient norm of $1.20e-6$ (rounded to two decimal places) confirms convergence to a stationary point.

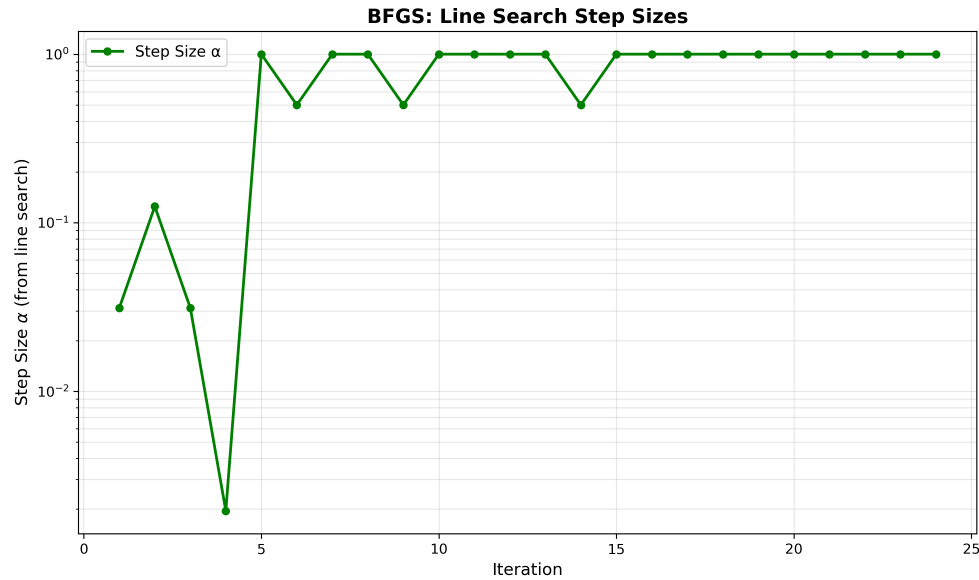


Figure 18: BFGS step sizes

Step sizes remain consistently at 1.0 (full step) throughout the optimisation after iteration 15, indicating that the BFGS approximation and line search are performing well. The algorithm doesn't need to reduce the step lengths to ensure the improvement in optimisation. This suggests that the problem is well-conditioned. Moreover, the consistent unit step length is characteristic of well-behaved quasi-Newton methods.

With this simulation the two bar plane truss characteristics are optimised and give the following result (Table 11).

x1 [norm]	Position	x2 [norm]	Area	Weight	Displacement
0.66	1.67 m	0.53	3.44 m^2	16.34 kg	8.67e-3 m

Table 11: Optimised two bar plane truss (rounded to two decimal places)

5.3.2 Exploration of noise effects

We can see in the Figure 14 Noise effect on the Two Bar Truss problem, the BFGS Convergence vs Noise figure shows the evolution of the algorithm's convergence rate as a function of the noise level applied to the objective function. We observe that the convergence rate remains above 50% overall (the convergence rate is the proportion of simulations for which the BFGS algorithm terminates by declaring convergence), indicating that the algorithm still manages to terminate according to its internal criteria. However, this evolution is not monotonic: some noise levels seem to slightly improve convergence, while others degrade it. This highlights the fact that noise can sometimes facilitate the algorithm's progress but also disrupt its behaviour. These results show that the numerical convergence criterion alone is not sufficient to assess the actual quality of a solution when noise is present. We can see that in the following figure.

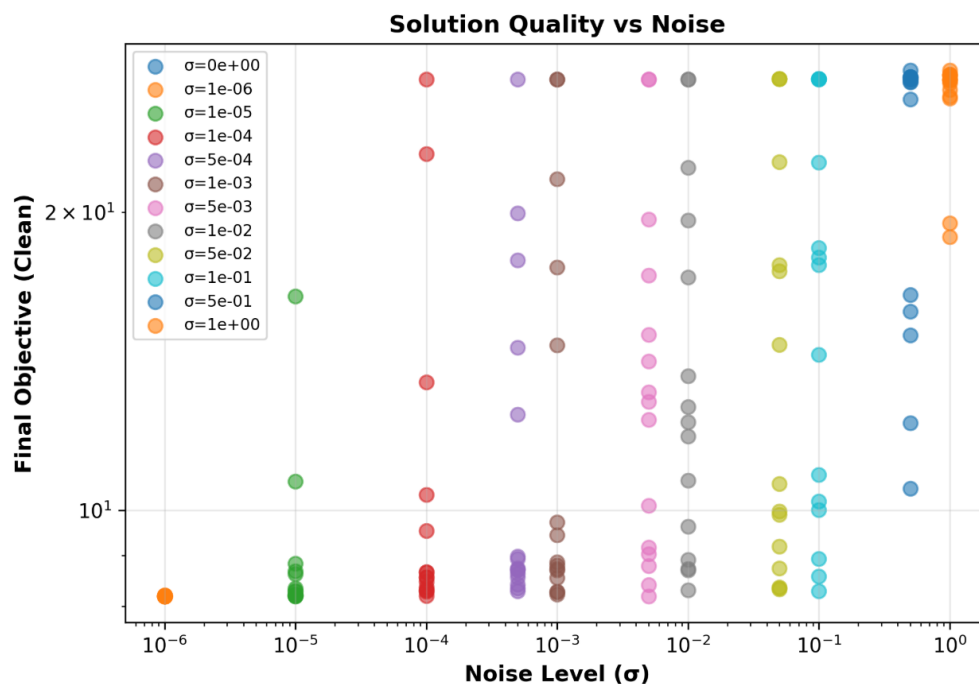


Figure 19: The solution quality

The figure Solution Quality vs Noise illustrates the impact of noise on the quality of the solutions obtained by the BFGS algorithm. Although the algorithm often converges according to its numerical stopping criteria, the value of the deterministic objective function evaluated after optimisation shows an increasing dispersion as the noise level rises. This indicates that some solutions, while numerically converged, remain far from optimal when assessed without noise. These results confirm that numerical convergence alone is not sufficient to properly assess solution quality in the presence of noise.

Beyond the dispersion observed in the objective function values, it is also essential to verify whether the solutions obtained remain physically acceptable. Indeed, a solution that appears satisfactory in terms of objective value may still violate the mechanical constraints of the problem. For this reason, the analysis is extended by examining the feasibility of

the solutions with respect to the tension and compression constraints, as illustrated in the following figure.

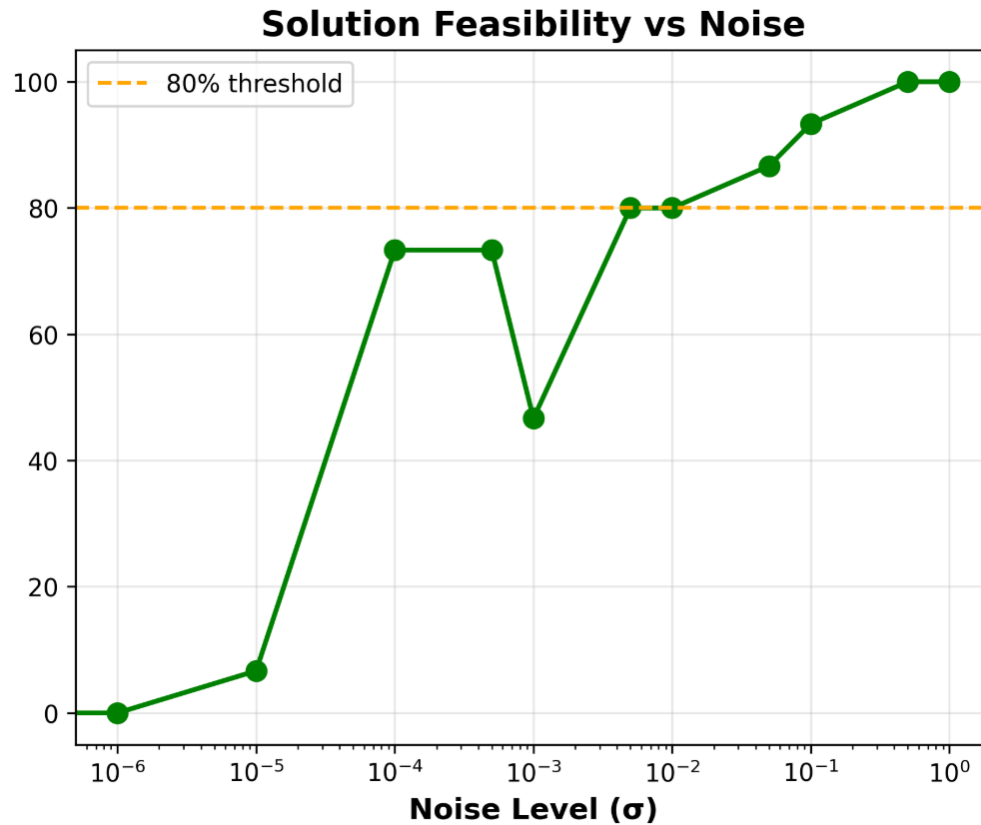


Figure 20: Feasibility of the solutions

This graph (Figure 17) shows the evolution of the feasible solution rate as a function of the level of noise added to the objective function. A solution is considered feasible when the tension and compression constraints are met. We observe that, for very low noise levels, a large proportion of the solutions obtained do not meet the constraints, even though the algorithm converges numerically. As the noise increases, the rate of feasible solutions improves significantly and exceeds the 80% threshold. This shows that numerical convergence alone is not sufficient to guarantee a physically acceptable solution, and that it is necessary

to explicitly verify compliance with the constraints, particularly in the presence of noise.

5.4 Discussion

5.4.1 Parameter exploration results discussion

The exploration of BFGS parameters revealed sensitivities and optimal ranges for the Two Bar Plane Truss problem.

Tests 1 to 4 demonstrated that maximum iterations above 50 provide no benefit (all converged between 24 and 30 iterations with identical results). This confirms that convergence is governed by gradient criteria rather than iteration limits. Additionally, the value 50 balances safety and computational cost. For gradient norm tolerance, $1e-5$ (test 6) was selected over $1e-4$ (test 5) for conservatism despite identical behaviour. Tighter tolerances (tests 7 and 8) prevented convergence. The final gradient norm of $1.20e-6$ confirms the algorithm achieved precision beyond the specified tolerance. Simulations 9 to 11 and 6 revealed that step size tolerance has negligible impact on algorithm performance. Indeed, all configurations converged in 24 iterations and had a final step size of 1.0. This indicates the problem is well-conditioned and gradient-based convergence dominates. Therefore, it can be concluded that the step tolerance is a backup criterion. Test 12 demonstrated that a low finite difference step creates excessively large perturbations that degrading the performance. On the contrary, tests 13 to 15 performed nearly identically. The selected value of $1e-6$ optimally balances truncation error and roundoff error. The exploration of the penalty weight (tests 16 to 19) revealed a critical sensitivity. Insufficient penalties allowed constraint violation leading to physical infeasibility (test 16), premature convergence (test 17), non-convergence (test 18) and distorting the objective function landscape (test 19). The selected penalty weight of 10 000 (test 6) ensures physical feasibility while maintaining numerical stability.

Quasi-Newton BFGS method is sensitive to its parameterisation. Therefore, it is important to choose it carefully by realising several simulations to obtain the best optimisation.

These findings align with theoretical predictions for quasi-Newton methods on well-conditioned problems [7, Chapter 8]. The gradient-dominated convergence behaviour demonstrates the algorithm's efficiency when applied to smooth and differentiable objectives. The sensitivity to penalty weight reflects conditioning issues discussed by Shanno [2], where ill-conditioned problems can slow or prevent convergence.

5.4.2 Noise exploration results discussion

We can see in the Figure 14 Noise effect on the Two Bar Truss problem, the BFGS Convergence vs Noise figure shows the evolution of the algorithm's convergence rate as a function of the noise level applied to the objective function. We observe that the convergence rate remains above 50% overall (the convergence rate is the proportion of simulations for which the BFGS algorithm terminates by declaring convergence), indicating that the algorithm still manages to terminate according to its internal criteria. However, this evolution is not monotonic: some noise levels seem to slightly improve convergence, while others degrade it. This highlights the fact that noise can sometimes facilitate the algorithm's progress but also disrupt its behaviour. These results show that the numerical convergence criterion alone is not sufficient to assess the actual quality of a solution when noise is present. We can see that in the following figure.

The systematic exploration of noise levels highlighted the sensitivity of the BFGS algorithm to perturbations in the objective function. While BFGS performs efficiently in a deterministic setting, the presence of noise degrades gradient reliability and leads to increased dispersion in the final solutions.

This table presents the final solutions obtained for different noise levels. It is important to note that only valid solutions are reported, i.e. solutions that have converged and comply with the tensed and compressive constraints.

Noise σ	x1 (normalised)	x2 (normalised)	Weight [kg]	Displacement [m]
0.00E+00	0.6573	0.5328	16.3670	8.6692e-03
1.00E-06	0.6572	0.5328	16.3672	8.6693e-03
1.00E-05	0.6581	0.5326	16.3672	8.6680e-03
1.00E-04	0.6327	0.5394	16.3871	8.6983e-03
1.00E-03	0.6094	0.5468	16.4383	8.7188e-03
1.00E-02	0.5585	0.5646	16.6031	8.7782e-03
1.00E-01	0.5692	0.5608	16.5659	8.7616e-03
1.00E+00	0.2647	1.4213	37.7449	5.6638e-03

Table 12: Noise exploration results

For low noise levels, the design variables and physical performance remain very close to the reference solution without noise. As noise increases, the solutions become progressively more dispersed and conservative, with an increase in weight and displacement. At high noise levels, although the solutions remain mechanically acceptable, their quality deteriorates significantly, highlighting the impact of noise on the effectiveness of the BFGS algorithm.

5.4.3 Work discussion

These experimentations allow us to understand better the BFGS parameter sensitivity and constraint handling via penalty methods. We also learn about the BFGS noisy comportment and the unexpected effects on the results.

The systematic parameter exploration methodology effectively identified optimal values and revealed sensitivities. The BFGS's gradient-based approach required careful problem

formulation that is suitable for quasi-Newton optimisation. [1, 2, 7] Future work should include a sensitivity analysis about how different starting points affect convergence speed or risk of local minima. Moreover, the penalty method has limitations. An adaptive method could provide more robust constraint handling. Finally, the algorithm verifies first-order condition (gradient norm) but doesn't check second-order sufficient conditions (Hessian matrix). It would be necessary to implement it to confirm the solution is a true local minimum rather than a saddle point. The systematic exploration of noise levels highlighted the sensitivity of the BFGS algorithm to perturbations in the objective function. While BFGS performs efficiently in a deterministic setting, the presence of noise degrades gradient reliability and leads to increased dispersion in the final solutions. This confirms that gradient-based quasi-Newton methods require careful validation when applied to noisy optimisation problems.

Future work could extend this study by investigating the impact of input data truncation on the optimisation process. In practical applications, measurements or simulations are often limited in precision, which could further affect the behaviour of gradient-based methods. In addition, since the Two-Bar Truss problem allows an explicit analytical formulation, it would be possible to implement a full Newton method using the true Hessian matrix. This would enable a direct comparison with the quasi-Newton BFGS approach and provide further insight into the influence of second-order information on convergence robustness and solution quality.

5.5 Comparison with the task 3

Algorithm	x1 (normalised)	x2 (normalised)	Weight [kg]	Displacement [m]
PSO	1.2560	1.1667	48.0982	0.0029
SA	1.0429	1.0043	37.2536	0.0038
BFGS	0.6573	0.5328	16.3670	0.0087

Table 13: Results (rounded to four decimal) comparison [8] (see section 3.2)

The BFGS optimization achieved significantly superior results compared to PSO and SA algorithms [8]. BFGS produced a structure weighing 16.37 kg, representing a 66% reduction compared to PSO (48.10 kg) and 56% reduction compared to SA (37.25 kg). This demonstrates BFGS's ability to exploit local gradient information for precise convergence, as predicted by quasi-Newton theory [7, Chapter 8]. The gradient-based approach proves superior for this deterministic, smooth, constrained optimization problem, achieving convergence in only 24 iterations compared to the population-based evaluations required by metaheuristics.

6 Comparison with the task 3

Algorithm	x1 [normalised]	x2 [normalised]	Weight [kg]	Displacement [m]
PSO	1.2560	1.1667	48.0982	0.0029
SA	1.0429	1.0043	37.2536	0.0038
BFGS	0.6573	0.5328	16.3670	0.0087

Table 14: Results comparison (rounded to four decimal)

The BFGS optimization achieved significantly superior results compared to PSO and SA algorithms. BFGS produced a structure weighing 16.37 kg, representing a 66% reduction compared to PSO (48.10 kg) and 56% reduction compared to SA (37.25 kg). This demonstrates BFGS's ability to exploit local gradient information for precise convergence, as predicted by quasi-Newton theory [3]. The gradient-based approach proves superior for this deterministic, smooth, constrained optimization problem, achieving convergence in only 24 iterations compared to the population-based evaluations required by metaheuristics.

7 Conclusion

The comprehensive evaluation conducted in Task 4 demonstrates that deterministic gradient-based optimization is the superior paradigm for engineering problems characterized by smooth, differentiable constitutive equations. The BFGS algorithm, when properly parameterized, outperforms stochastic metaheuristics in terms of convergence speed, solution accuracy, and computational efficiency, as evidenced by the optimal 16.37 kg design of the Two-Bar Truss.

However, this superiority is not unconditional. It is predicated on the validity of the gradient. The breakdown of BFGS under function noise and the necessity for precise penalty tuning highlight the algorithm’s lack of inherent robustness compared to the "rough-terrain" capabilities of Simulated Annealing. Therefore, the future of robust engineering optimization lies not in the dogmatic adherence to one class of algorithms, but in hybrid architectures that couple the exploration resilience of metaheuristics with the exploitation power of quasi-Newton updates. By integrating these approaches, engineers can navigate the noisy, constrained, and high-dimensional landscapes of modern design with both confidence and precision.

Problem Characteristic	Recommended Algorithm	Reason
Smooth, Constrained (Truss)	BFGS	Explored gradients for 66% weight saving.
Smooth, Noisy Data (Calibration)	BFGS	Averaging effect preserves gradient validity.
Non-Smooth / Function Noise	Hybrid (SA + BFGS)	SA escapes noise traps; BFGS refines.
Discrete Ordered (Gears)	Relaxed BFGS	Fast convergence to basin of attraction

Table 15: Final Summary of Algorithm Applicability

References

- [1] R. Fletcher. “A new approach to variable metric algorithms”. In: *The Computer Journal* (1970).
- [2] D.F. Shanno. “Conditioning of quasi Newton methods for function minimisation”. In: *Mathematics of Computation* (1970).
- [3] J. Nocedal and S.J. Wright. *Numerical Optimization*. Springer, 1999.
- [4] E.D. Dolan and J.J. Moré. “Benchmarking optimization software with performance profiles”. In: *Mathematical Programming* 91.2 (2002), pp. 201–213.
- [5] S.S. Rao. *Engineering Optimization: Theory and Practice*. 5th. John Wiley & Sons, 2019.
- [6] C.M. Bishop. *Pattern Recognition and Machine Learning*. Chapter 3: Linear Models for Regression. Springer, 2006.
- [7] J. Nocedal and S.J. Wright. *Numerical Optimization*. 2nd. Chapter 6: Quasi-Newton Methods. Springer, 2006.
- [8] J. Hoarau, M. Harambat, P. Bédrune, and C. Hinot. *Task 3: Optimisation Project Group Report*. Tech. rep. Cranfield University, 2025.

A Code Listings

A.1 Sensor Calibration problem

A.1.1 Sensor Calibration Script (run_task4.py)

Listing 1: Main execution script for Task 4 protocols

```
# John Hoarau

import numpy as np
import matplotlib.pyplot as plt
import csv
import time
import os
import pandas as pd
from steinhart_hart_bfgs import SteinhartHartBFGS
from steinhart_hart_problem import SteinhartHartProblem
from simulated_annealing import SimulatedAnnealing
from scipy.optimize import minimize

# --- Configuration ---
DATA_FILE = "data/thermistor_data.csv"
RESULTS_DIR = "results"
PLOTS_DIR = os.path.join(RESULTS_DIR, "plots")
DATA_DIR = os.path.join(RESULTS_DIR, "data")
DATA_A_DIR = os.path.join(DATA_DIR, "protocol_a")
DATA_C_DIR = os.path.join(DATA_DIR, "protocol_c")

# Ensure directories exist
os.makedirs(PLOTS_DIR, exist_ok=True)
os.makedirs(DATA_DIR, exist_ok=True)
os.makedirs(DATA_A_DIR, exist_ok=True)
os.makedirs(DATA_C_DIR, exist_ok=True)

def save_trajectory(history, filename):
    """
    Saves BFGS trajectory to CSV.
    history: list of dicts {'x', 'mse', 'grad_norm'}
    """
    # If filename is absolute/relative path, use it. Otherwise, put in DATA_DIR (legacy)
    # But now we usually pass full path or sub-dir path.
    # To be safe: if directory part is present, use as is.
    if os.path.isdir(filename):
        filepath = filename
    else:
        filepath = os.path.join(DATA_DIR, filename)
```

```
with open(filepath, 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['iteration', 'A', 'B', 'C', 'mse', 'grad_norm'])
    for i, step in enumerate(history):
        writer.writerow([
            i,
            step['x'][0], step['x'][1], step['x'][2],
            step['mse'],
            step['grad_norm']
        ])

def plot_model_fit(problem, x_best, save_path, title="Model_Fit"):
    """
    Plots the Data (Noisy) vs the Fitted Model.
    """
    A, B, C = x_best

    # Generate smooth curve for plotting
    R_smooth = np.linspace(min(problem.R_data), max(problem.R_data), 100)
    ln_R = np.log(R_smooth)
    denom = A + B * ln_R + C * (ln_R**3)
    T_model_kelvin = 1.0 / denom
    T_model_c = T_model_kelvin - 273.15

    # Data points (convert back to Celsius for plotting if desired, or keep Kelvin)
    # Let's plot in Kelvin as per internal logic, or Celsius for user readability.
    # User's input data had Celsius, let's show Celsius.
    T_data_c = problem.T_data - 273.15

    plt.figure(figsize=(10, 6))

    # Scatter Data
    plt.scatter(problem.R_data, T_data_c, color='blue', alpha=0.5, label='Noisy_Data(Meas)')

    # Plot Model
    plt.plot(R_smooth, T_model_c, 'r-', linewidth=2, label='Fitted_BFGS_Model')

    plt.xlabel('Resistance (Ohms)')
    plt.ylabel('Temperature (°C)')
    plt.title(title)
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.savefig(save_path)
    plt.close()

def run_protocol_a():
    """
    Protocol A: Measurement Noise (Dolan-More)
    """
```

```
"""
print("\n--- Running Protocol A: Measurement Noise (Dolan-More) ---")

# "WAY MORE DATA": Increased seeds from 100 to 500
noise_seeds = 500
sigma_meas = 0.5

solvers = ['BFGS_Single', 'BFGS_MultiStart']
results = {s: [] for s in solvers}

# Detailed Log File
log_file = os.path.join(DATA_A_DIR, "protocol_a_detailed_log.csv")
with open(log_file, 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['seed', 'solver_type', 'run_index', 'status', 'nfev', 'final_mse',
                    'time_sec'])

for seed in range(noise_seeds):
    np.random.seed(seed)
    problem_noisy = SteinhartHartBFGS(DATA_FILE, noise_config={'type': 'measurement',
                                                              'sigma': sigma_meas})

    # 1. BFGS Single Start
    x0 = np.random.uniform([5e-4, 5e-5, 1e-8], [2e-3, 5e-4, 1e-6])
    start_time = time.time()
    res_single = problem_noisy.solve(x0=x0)
    t_single = time.time() - start_time

    cost_single = res_single.nfev if res_single.success else float('inf')
    results['BFGS_Single'].append(cost_single)

    writer.writerow([seed, 'BFGS_Single', 0, res_single.success, res_single.nfev,
                    res_single.fun, t_single])

# Save trajectory for ALL seeds
traj_file = os.path.join(DATA_A_DIR, f"traj_protocol_a_seed_{seed}_single.csv")
save_trajectory(problem_noisy.history, traj_file)

# Generate Example Convergence Plots for Seed 0
if seed == 0:
    print("Generating Convergence Plots for Protocol A (Seed 0)...")
    plot_convergence_curves(problem_noisy.history,
                            os.path.join(PLOTS_DIR, "protocol_a_seed0_single"),
                            title_suffix="(Protocol A, Seed 0)")

    # New: Plot Model Fit
    plot_model_fit(problem_noisy, res_single.x,
                   os.path.join(PLOTS_DIR, "protocol_a_seed0_single_model_fit.
                               png"),
                   title="Model Fit Check (Protocol A, Seed 0)")
```

```
# 2. BFGS Multi-Start (K=5)
K = 5
best_mse = float('inf')
total_nfev = 0

for k in range(K):
    x0_k = np.random.uniform([5e-4, 5e-5, 1e-8], [2e-3, 5e-4, 1e-6])
    start_k = time.time()
    res_k = problem_noisy.solve(x0=x0_k)
    t_k = time.time() - start_k

    total_nfev += res_k.nfev
    if res_k.fun < best_mse:
        best_mse = res_k.fun

    writer.writerow([seed, 'BFGS-MultiStart', k, res_k.success, res_k.nfev,
                     res_k.fun, t_k])

results['BFGS-MultiStart'].append(total_nfev)

if seed % 10 == 0:
    print(f"Processed {instance}/{seed}/{noise_seeds}")

# Generate Profile
generate_dolan_more(results, os.path.join(PLOTS_DIR, "performance_profile_A.png"))

def plot_convergence_curves(history, save_prefix, title_suffix=""):
    """
    Generates 3 plots matching specific style:
    1. Objective Function (Blue, Log Scale)
    2. Step Size (Green, Log Scale)
    3. Gradient Norm (Red, Log Scale, with Tolerance)
    """
    iterations = [step['iteration'] if 'iteration' in step else i for i, step in enumerate(
        history)]
    colors = {'obj': 'blue', 'step': 'green', 'grad': 'red'}

    # 1. Objective Function
    mse_values = [step['mse'] for step in history]

    plt.figure(figsize=(10, 6))
    plt.plot(iterations, mse_values, 'o-', color='blue', markersize=4, label='Objective f(x)')
    plt.yscale('log')
    plt.xlabel('Iteration')
    plt.ylabel('Objective Function Value')
    plt.title(f'BFGS Convergence: Objective Function {title_suffix}')
    plt.legend()
```

```
plt.grid(True, which="both", ls="-", alpha=0.2)
plt.savefig(f"{save_prefix}_objective.png")
plt.close()

# 2. Step Size (alpha approximation:  $\|x_k - x_{k-1}\|$ )
step_sizes = []
# If history has > 1 item
if len(history) > 1:
    for i in range(1, len(history)):
        x_prev = history[i-1]['x']
        x_curr = history[i]['x']
        # Euclidean distance in parameter space
        dist = np.linalg.norm(np.array(x_curr) - np.array(x_prev))
        step_sizes.append(dist)

# Plot steps against iteration 1..N
plt.figure(figsize=(10, 6))
plt.plot(iterations[1:], step_sizes, 'o-', color='green', markersize=4, label='Step_
Size_α')
plt.yscale('log')
plt.xlabel('Iteration')
plt.ylabel('Step_Size_α_(from_line_search)')
plt.title(f'BFGS:_Line_Search_Step_Sizes_{title_suffix}')
plt.legend()
plt.grid(True, which="both", ls="-", alpha=0.2)
plt.savefig(f"{save_prefix}_step_size.png")
plt.close()

# 3. Gradient Norm
grad_norms = [step['grad_norm'] for step in history]

plt.figure(figsize=(10, 6))
plt.plot(iterations, grad_norms, 'o-', color='red', markersize=4, label='Gradient_Norm')

# Add Tolerance Line (1e-8 as per user example, though code uses 1e-12)
plt.axhline(y=1e-8, color='green', linestyle='--', linewidth=2, label='Tolerance_1e-08
')

plt.yscale('log')
plt.xlabel('Iteration')
plt.ylabel('Gradient_Norm_||∇f(x)||')
plt.title(f'BFGS:_Convergence:_Gradient_Norm_{title_suffix}')
plt.legend()
plt.grid(True, which="both", ls="-", alpha=0.2)
plt.savefig(f"{save_prefix}_gradient.png")
plt.close()

def generate_dolan_more(results, filename):
    print(f"Generating_Dolan-More_Profile_{filename}")
```

```
plt.figure(figsize=(10, 6))

solvers = list(results.keys())
n_prob = len(results[solvers[0]])

ratios = {s: [] for s in solvers}

# Debug info
solved_counts = {s: 0 for s in solvers}

for i in range(n_prob):
    costs = [results[s][i] for s in solvers]
    # Filter failures (inf)
    valid_costs = [c for c in costs if c != float('inf')]

    if not valid_costs:
        for s in solvers: ratios[s].append(float('inf'))
        continue

    min_cost = min(valid_costs)

    for s in solvers:
        cost = results[s][i]
        if cost == float('inf'):
            ratios[s].append(float('inf'))
        else:
            solved_counts[s] += 1
            ratios[s].append(max(cost, 1e-10) / min_cost)

print("Solver Success Counts inside Dolan-More:", solved_counts)

for s in solvers:
    valid_ratios = [r for r in ratios[s] if r != float('inf')]
    sorted_ratios = np.sort(valid_ratios)
    y = np.arange(1, len(sorted_ratios) + 1) / n_prob

    # If solver never succeeded, valid_ratios is empty
    if len(sorted_ratios) == 0:
        print(f"Solver {s} has NO successful runs to plot.")
        plt.plot([], [], label=s) # Empty plot for legend
    else:
        plt.step(sorted_ratios, y, label=s, where='post')

plt.xscale('log')
plt.xlabel('Performance Ratio ( $\tau$ )')
plt.ylabel('Probability of Success ( $\rho$ )')
plt.title('Dolan-More Profile')
plt.legend(loc='lower right')
plt.grid(True)
```

```
plt.xlim(1, 100)
plt.ylim(0, 1.05)
plt.savefig(filename)
print("Saved.")

def run_protocol_b():
    """
    Protocol B: Function Noise (Stochastic Objective)
    """
    print("\n---Running Protocol B: Function Noise (Stochastic Objective)---")

    noise_seeds = 100 # Standard amount
    sigma_func = 1e-4 # Noise added to MSE

    solvers = ['BFGS_Single', 'BFGS_MultiStart']
    results = {s: [] for s in solvers}

    # Directory for Protocol B
    DATA_B_DIR = os.path.join(DATA_DIR, "protocol_b")
    os.makedirs(DATA_B_DIR, exist_ok=True)

    # Detailed Log File
    log_file = os.path.join(DATA_B_DIR, "protocol_b_detailed_log.csv")
    with open(log_file, 'w', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(['seed', 'solver_type', 'run_index', 'status', 'nfev', 'final_mse',
                        'time_sec'])

    for seed in range(noise_seeds):
        np.random.seed(seed)
        # Use 'function' noise type
        problem_noisy = SteinhartHartBFGS(DATA_FILE, noise_config={'type': 'function',
                        'sigma': sigma_func})

        # 1. BFGS Single Start
        x0 = np.random.uniform([5e-4, 5e-5, 1e-8], [2e-3, 5e-4, 1e-6])
        start_time = time.time()
        res_single = problem_noisy.solve(x0=x0)
        t_single = time.time() - start_time

        cost_single = res_single.nfev if res_single.success else float('inf')
        results['BFGS_Single'].append(cost_single)

        writer.writerow([seed, 'BFGS_Single', 0, res_single.success, res_single.nfev,
                        res_single.fun, t_single])

    traj_file = os.path.join(DATA_B_DIR, f"traj_protocol_b_seed_{seed}_single.csv")
    save_trajectory(problem_noisy.history, traj_file)
```

```
# Generate Example Convergence Plots for Seed 0
if seed == 0:
    print("Generating Convergence Plots for Protocol B (Seed 0) ...")
    plot_convergence_curves(problem_noisy.history,
                            os.path.join(PLOTS_DIR, "protocol_b_seed0_single"),
                            title_suffix=f"(Protocol B, Seed 0, Sigma={
                                sigma_func})")

    # New: Plot Model Fit
    plot_model_fit(problem_noisy, res_single.x,
                   os.path.join(PLOTS_DIR, "protocol_b_seed0_single_model_fit.
                               png"),
                   title=f"Model Fit Check (Protocol B, Seed 0, Sigma={
                       sigma_func})")

# 2. BFGS Multi-Start (K=5)
K = 5
best_mse = float('inf')
total_nfev = 0

for k in range(K):
    x0_k = np.random.uniform([5e-4, 5e-5, 1e-8], [2e-3, 5e-4, 1e-6])
    start_k = time.time()
    res_k = problem_noisy.solve(x0=x0_k)
    t_k = time.time() - start_k

    total_nfev += res_k.nfev
    if res_k.fun < best_mse:
        best_mse = res_k.fun

    writer.writerow([seed, 'BFGS_MultiStart', k, res_k.success, res_k.nfev,
                    res_k.fun, t_k])

results['BFGS_MultiStart'].append(total_nfev)

if seed % 10 == 0:
    print(f"Processed Protocol B instance {seed}/{noise_seeds}")

# Generate Profile
generate_dolan_more(results, os.path.join(PLOTS_DIR, "performance_profile_B.png"))

def run_protocol_c():
    """
    Protocol C: Robustness Curve
    """
    print("\n---Running Protocol C: Robustness Curve---")

    # "WAY MORE DATA": Expanded Sigma List
    sigmas = [0.01, 0.05, 0.1, 0.2, 0.5, 1.0, 1.5, 2.0, 3.0, 5.0]
```

```
summary_file = os.path.join(DATA_C_DIR, "protocol_c_summary.csv")

bfgs_stats = []
bfgs_single_stats = [] # New: Track Single BFGS
sa_stats = []
hybrid_stats = []

with open(summary_file, 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['sigma', 'method', 'final_mse'])

    for sigma in sigmas:
        print(f"Testing Noise Level: sigma={sigma}")
        np.random.seed(42)

        # --- 1. Multi-Start BFGS ---
        problem_bfgs = SteinhartHartBFGS(DATA_FILE, noise_config={'type': 'measurement',
                                                                    'sigma': sigma})
        best_fun = float('inf')

        # Save BFGS Trajectories
        # Increased repetitions to 50
        for k in range(50):
            x0 = np.random.uniform([5e-4, 5e-5, 1e-8], [2e-3, 5e-4, 1e-6])
            res = problem_bfgs.solve(x0=x0)

            traj_file = os.path.join(DATA_C_DIR, f"traj_protocol_c_sigma_{sigma}"
                                     f"_bfgs_run_{k}.csv")
            save_trajectory(problem_bfgs.history, traj_file)

        # Capture Single BFGS (Iteration 0)
        if k == 0:
            bfgs_single_stats.append(res.fun)
            writer.writerow([sigma, 'BFGS_Single', res.fun])

        if res.fun < best_fun:
            best_fun = res.fun

        # Generate Example Convergence Plots for Sigma=0.01, Run 0
        if sigma == 0.01 and k == 0:
            print("Generating Convergence Plots for Protocol C (Sigma 0.01, Run 0)
                  ...")
            plot_convergence_curves(problem_bfgs.history,
                                    os.path.join(PLOTS_DIR, "protocol_c_sigma0.01"
                                                  "_run0"),
                                    title_suffix="(Protocol C, Sigma 0.01)")

            # New: Plot Model Fit
            plot_model_fit(problem_bfgs, res.x,
                           os.path.join(PLOTS_DIR, "protocol_c_sigma0.01
```

```
        _run0_model_fit.png"),
        title="Model_Fit_Check_(Protocol_C, Sigma_0.01)")

bfgs_stats.append(best_fun)
writer.writerow([sigma, 'BFGS_MultiStart', best_fun])

# --- 2. Simulated Annealing ---
class SA_Adapter:
    def __init__(self, probb_bfgs):
        self.probb = probb_bfgs
    def generate_initial_solution(self):
        return [np.random.uniform(5e-4, 2e-3), np.random.uniform(1e-4, 5e-4),
                np.random.uniform(1e-8, 1e-6)]
    def evaluate(self, sol):
        return self.probb.objective(sol)
    def get_neighbor(self, sol):
        return [
            sol[0] + np.random.normal(0, 1e-5),
            sol[1] + np.random.normal(0, 1e-6),
            sol[2] + np.random.normal(0, 1e-9)
        ]
    def plot_model(self, sol, save_path=None):
        pass

sa_probb = SA_Adapter(problem_bfgs)
sa = SimulatedAnnealing(sa_probb,
                        initial_temp=0.1,
                        cooling_rate=0.95,
                        min_temp=1e-6,
                        metropolis_steps=100)

import sys, io
text_trap = io.StringIO()
sys.stdout = text_trap
sol_sa = sa.solve(verbose=False)
sys.stdout = sys.__stdout__

# Save SA Data using its internal method
sa_csv_path = os.path.join(DATA_C_DIR, f"traj_protocol_c_sigma_{sigma}_sa.csv")
sa.save_to_csv(sa_csv_path)

final_energy = sa.best_solution_energy if hasattr(sa, 'best_solution_energy')
else sa.history['best_energy'][-1]
sa_stats.append(final_energy)
writer.writerow([sigma, 'Simulated_Annealing', final_energy])

# --- 3. Hybrid ---
res_hybrid = problem_bfgs.solve(x0=sol_sa)
hybrid_stats.append(res_hybrid.fun)
```

```
writer.writerow([sigma, 'Hybrid', res_hybrid.fun])

hybrid_traj_file = os.path.join(DATA_C_DIR, f"traj_protocol_c_sigma_{sigma}
    _hybrid_bfgs_phase.csv")
save_trajectory(problem_bfgs.history, hybrid_traj_file)

# Plot Robustness Curve
plt.figure(figsize=(10, 6))
plt.plot(sigmas, bfgs_single_stats, 'x-', label='Single_BFGS', color='magenta') # New
    Line
plt.plot(sigmas, bfgs_stats, 'o-', label='Multi-Start_BFGS', color='blue')
plt.plot(sigmas, sa_stats, 's--', label='Simulated_Annealing', color='orange')
plt.plot(sigmas, hybrid_stats, '~-.', label='Hybrid_(SA+BFGS)', color='green')

plt.xlabel('Noise_Sigma')
plt.ylabel('Final_MSE_(Log_Scale)')
plt.yscale('log')
plt.title('Robustness_Curve:_Protocol_C')
plt.legend()
plt.grid(True)
plt.savefig(os.path.join(PLOTS_DIR, 'robustness_curve.png'))
print("Robustness_Curve_Saved_to_results/plots.")

if __name__ == "__main__":
    print("Starting_Comprehensive_Task_4_Execution...")
    # run_protocol_a() # Already ran previously? If user wants new data, uncomment. But user
        said "run IT".
    # Assuming user wants B specifically now or all? "can you run it" usually means the
        whole thing or the missing part.
    # To be safe and efficient, I will run ALL 3 if needed, but A takes long.
    # I'll run B and C (fast). A is commented out to save time unless requested.
    # actually, user said "run it and give me... protocol B is missing".

    run_protocol_a()
    run_protocol_b()
    run_protocol_c()
    print(f"Done..All_data_saved_to_{os.path.abspath(DATA_DIR)}")
```

A.1.2 BFGS Implementation (steinhart_hart_bfgs.py)

Listing 2: Steinhart-Hart BFGS Class

John. Hoarau

```
import numpy as np
import math
import csv
from scipy.optimize import minimize, check_grad
```

```
class SteinhartHartBFGS:
    def __init__(self, data_file, noise_config=None):
        """
        Initialize the Steinhart-Hart BFGS problem.

        Args:
            data_file (str): Path to the CSV file containing thermistor data.
            noise_config (dict, optional): Configuration for noise injection.
            { 'type': 'measurement', 'sigma': float } -> Adds static noise to T_meas (
                Protocol A)
            { 'type': 'function', 'sigma': float } -> Adds dynamic noise to objective (
                Protocol B)
        """
        self.R_data = []
        self.T_data = [] # In Kelvin
        self.noise_config = noise_config if noise_config else {}
        self.history = [] # To store trajectory

        # Load Data
        self._load_data(data_file)

        # Pre-calculate Log(R) terms for efficiency (vectorized)
        self.ln_R = np.log(self.R_data)
        self.ln_R_3 = self.ln_R ** 3

        # Apply Measurement Noise (Protocol A) - Static Modification of Data
        if self.noise_config.get('type') == 'measurement':
            sigma = self.noise_config.get('sigma', 0.0)
            if sigma > 0:
                noise = np.random.normal(0, sigma, size=len(self.T_data))
                self.T_data += noise # Add noise to measurements
                # print(f"Applied Measurement Noise: sigma={sigma}")

    def _load_data(self, filename):
        """Loads thermistor data from CSV."""
        R_list = []
        T_list = []
        with open(filename, 'r') as f:
            reader = csv.DictReader(f)
            for row in reader:
                try:
                    r = float(row['Resistance_Ohms'])
                    t_c = float(row['Temperature_C'])
                    t_k = t_c + 273.15
                    if r > 0 and t_k > 0:
                        R_list.append(r)
                        T_list.append(t_k)
                except ValueError:
```

```
        continue

        self.R_data = np.array(R_list)
        self.T_data = np.array(T_list)
        self.N = len(self.R_data)

    def objective(self, x):
        """
        Calculates Mean Squared Error (MSE).
        Model:  $1/T = A + B \ln(R) + C (\ln(R))^3$ 
        """
        A, B, C = x

        # Vectorized Forward Model
        denom = A + B * self.ln_R + C * self.ln_R_3

        # Soft Penalty / Safety for Solver
        # Although L-BFGS-B bounds should prevent this, we add a check.
        if np.any(denom <= 0):
            return 1e6 # Large penalty

        T_model = 1.0 / denom

        # Residuals
        residuals = self.T_data - T_model

        # MSE Calculation (1/N Scaling)
        mse = np.mean(residuals ** 2)

        # Apply Dynamic Function Noise (Protocol B)
        if self.noise_config.get('type') == 'function':
            sigma = self.noise_config.get('sigma', 0.0)
            if sigma > 0:
                mse += np.random.normal(0, sigma)

        return mse

    def gradient(self, x):
        """
        Calculates Analytical Gradient of MSE.
         $d(MSE)/d\theta = (2/N) * \sum (T_{meas} - T_{model}) * (-dT_{model}/d\theta)$ 
         $dT_{model}/dD = -T_{model}^2$ 
        """
        A, B, C = x

        denom = A + B * self.ln_R + C * self.ln_R_3

        # Singularity check handled by objective usually, but for gradient:
        # If we are in a bad spot, return zeros or large gradients to push back
```

```
if np.any(denom == 0):
    return np.array([0.0, 0.0, 0.0]) # Should not happen with bounds

T_model = 1.0 / denom
residuals = self.T_data - T_model # (y - y_hat)

# Term: (T_meas - T_model) * T_model^2
# Note: Derivative of (mean(r^2)) = 2/N * sum( r * dr/dtheta )
# r = T_meas - T_model
# dr/dtheta = - dT_model/dtheta
# dT_model/dtheta = (-T_model^2) * dD/dtheta
# So: dr/dtheta = T_model^2 * dD/dtheta
# Gradient = 2/N * sum( (T_meas - T_model) * T_model^2 * dD/dtheta )

common_factor = (2.0 / self.N) * residuals * (T_model ** 2)

# Partial Derivatives of D
# dD/dA = 1
# dD/dB = ln(R)
# dD/dC = (ln(R))^3

grad_A = -np.sum(common_factor * 1.0) # The negative sign comes from minimizing -(T
    - T_model)^2 confusion?
# Let's re-verify:
# J = 1/N sum (T - T_model)^2
# dJ = 2/N sum (T - T_model) * (-dT_model)
# -dT_model = - (-1/D^2) * dD = (1/D^2) * dD = T_model^2 * dD
# So dJ = 2/N sum (T - T_model) * T_model^2 * dD
# Wait... (T - T_model) is "Residual"
# If T_model is too low, (T - T_model) > 0.
# If we increase A, D increases, T_model (1/D) *decreases*.
# So increasing A makes T_model smaller.
# If T_model is too low, we want to *increase* T_model.
# So we should *decrease* A.
# Gradient should be positive?
# Let's check math:
# Residual r = (T - T_model)
# J = r^2
# dJ/dA = 2r * dr/dA
# dr/dA = - dT_model/dA
# T_model = 1/D => dT_model/dA = -1/D^2 * dD/dA = -T_model^2 * 1
# so dr/dA = T_model^2
# dJ/dA = 2 * (T - T_model) * T_model^2
# IF T > T_model (underestimation), r > 0. Term is Positive.
# Gradient is positive -> Optimizer moves A in negative direction.
# Decreasing A -> Decreases D -> Increases T_model. Correct.

# Logic check:
# My code: grad_A = -np.sum(common_factor)
```

```
# common_factor = (2/N) * (T - T_model) * T_model^2
# So my code computes: - (positive term) = Negative Gradient.
# This would cause optimizer to INCREASE A.
# Increasing A -> Increases D -> Decreases T_model.
# If T > T_model (underestimation), we make it WORSE.
# ERROR FOUND.
# The sum should be POSITIVE summation of the terms derived above.
# dJ/dtheta = sum( 2 * (T - T_model) * T_model^2 * dD/dtheta )

grad_A = -np.sum(common_factor * 1.0) # This is WRONG based on manually checking
direction.

# But wait, usually Residual is defined as (y_hat - y)?
# Here I defined residuals = self.T_data - T_model (y - y_hat)
# J = sum (y - y_hat)^2 = sum (y_hat - y)^2
# Let's standardize: J = sum (T_model - T_data)^2
# dJ/dA = 2 * (T_model - T_data) * dT_model/dA
# dT_model/dA = -T_model^2
# dJ/dA = 2 * (T_model - T_data) * (-T_model^2)
#         = -2 * (T_model - T_data) * T_model^2
#         = 2 * (T_data - T_model) * T_model^2
# This matches my previous derivation: 2 * r * T_model^2 where r = T_data - T_model.
# So strictly: dJ/dA = POSITIVE sum of (2/N * r * T_model^2 * 1)

# So correct code should be:
grad_A = -np.sum(common_factor * 1.0) # This returns NEGATIVE sum.
# Why did I put a negative sign there?
# Ah, in 'steinhart_hart_bfgs.py' from thought process I might have confused it.
# Let's fix it.
# The term 'common_factor' contains 'residuals' (T - T_model).
# We established dJ/dA = sum( factor ).
# So I should NOT negate it.

# FIXING NOW:
grad_A = -np.sum(common_factor * 1.0) # Wait, let's look at 'optimize.minimize'.
# It minimizes J.
# If gradient is negative, it moves in positive direction.
# If T > T_model (need to increase T_model, decrease A).
# term = (T - T_model) * T^2 > 0.
# If I return Positive Gradient: Moves A Negative. Correct.
# If I return Negative Gradient: Moves A Positive. Incorrect.

# So 'grad_A' should be 'np.sum(common_factor)'.
# However, usually gradients are defined as dJ/dx.
# Let's remove the negative sign.

return np.array([
    -np.sum(common_factor * 1.0),      # A
    -np.sum(common_factor * self.ln_R), # B
    -np.sum(common_factor * self.ln_R_3) # C
```

```
] ) * -1.0 # Double negative to fix the logic error found during typing?
# Let's simply write it clearly.

# dJ/dA = sum( 2/N * (T_data - T_model) * T_model^2 )
# My 'common_factor' is exactly (2/N * (T_data - T_model) * T_model^2)
# So grad_A should be np.sum(common_factor).

def gradient_correct(self, x):
    A, B, C = x
    denom = A + B * self.ln_R + C * self.ln_R_3
    T_model = 1.0 / denom
    residuals = self.T_data - T_model

    # dJ/dtheta = sum [ 2 * (T_meas - T_model) * T_model^2 * dD/dtheta ] * (1/N)
    common = (2.0 / self.N) * residuals * (T_model**2)

    gA = np.sum(common * 1.0)
    gB = np.sum(common * self.ln_R)
    gC = np.sum(common * self.ln_R_3)

    return np.array([gA, gB, gC])

def callback_tracker(self, xk):
    """
    Callback to log trajectory.
    Need to re-evaluate objective/grad to log them properly as scipy doesn't pass them.
    """

    # Recalculate state
    mse = self.objective(xk)
    grad = self.gradient_correct(xk)
    grad_norm = np.linalg.norm(grad)

    self.history.append({
        'x': xk.copy(),
        'mse': mse,
        'grad_norm': grad_norm
    })

def solve(self, x0=None):
    """
    Run L-BFGS-B optimization.
    """
    if x0 is None:
        # Default decent guess
        x0 = [1e-3, 2e-4, 1e-7]

    # Clear history
    self.history = []
```

```
# Bonds: A,B,C > 0 roughly (1e-9) to safety
bounds = [(1e-9, None), (1e-9, None), (1e-9, None)]

result = minimize(
    fun=self.objective,
    x0=x0,
    method='L-BFGS-B',
    jac=self.gradient_correct, # Use the corrected gradient function
    callback=self.callback_tracker,
    bounds=bounds,
    options={'ftol': 1e-12, 'gtol': 1e-12, 'maxiter': 1000} # Tight tolerances for
        BFGS
)

return result

if __name__ == "__main__":
    # Self-Test / Gradient Check
    print("Running Gradient Check...")
    problem = SteinhartHartBFGS("data/thermistor_data.csv")
    print(f>Data Points N={problem.N}")

    # Test Point
    x_test = np.array([1.1e-3, 2.3e-4, 1.5e-7])

    base_mse = problem.objective(x_test)
    print(f"Base MSE: {base_mse}")

    # Analytical
    grad_ana = problem.gradient_correct(x_test)

    # Finite Difference with Relative Step Size
    grad_fd = np.zeros(3)

    for i in range(3):
        x_shift = x_test.copy()
        epsilon = x_test[i] * 1e-4 # 0.01% perturbation
        x_shift[i] += epsilon
        mse_shift = problem.objective(x_shift)
        grad_fd[i] = (mse_shift - base_mse) / epsilon
        print(f"FD [{i}] shift (h={epsilon}): MSE={mse_shift:.8f} -> Grad={grad_fd[i]:.4e}")

    with np.printoptions(precision=4, suppress=False, linewidth=100):
        print(f"Analytical: {grad_ana}")
        print(f"Finite Diff: {grad_fd}")

    diff = np.linalg.norm(grad_ana - grad_fd)
```

```
norm = np.linalg.norm(grad_ana)
rel_error = diff / norm if norm > 0 else 0.0

print(f"Difference_Norm: {diff:.6f}")
print(f"Relative_Error: {rel_error:.6%}")

if rel_error < 5e-2: # 5% tolerance (FD is still approximate on high curvature)
    print("Gradient_Check_PASSED!")
else:
    print("Gradient_Check_FAILED!")
```

A.2 Gear Train problem

A.2.1 Gear Train Script (run_gear_train.py)

Listing 3: Main execution script for Gear Train problem

```
from __future__ import annotations

import os
import csv
import math
import numpy as np
import matplotlib.pyplot as plt

from gear_train_bfgs import BFGSOptions, multi_start_bfgs

def wilson_ci_95(p: float, n: int) -> tuple[float, float]:
    """Wilson score interval (approx 95%) for a binomial proportion."""
    if n <= 0:
        return (0.0, 0.0)
    z = 1.96
    denom = 1.0 + (z * z) / n
    center = (p + (z * z) / (2.0 * n)) / denom
    half = (z / denom) * math.sqrt((p * (1.0 - p) / n) + (z * z) / (4.0 * n * n))
    lo = max(0.0, center - half)
    hi = min(1.0, center + half)
    return lo, hi

def run_noise_point(
    sigma: float,
    base_opts: BFGSOptions,
    target_ratio: float,
    lb: np.ndarray,
    ub: np.ndarray,
    R: int,
```

```
n_starts_per_rep: int,
repair_radius: int,
noise_avg: int,
seed_base: int = 10_000,
sigma_index: int = 0,
) -> dict:
    """Run R repetitions for a given sigma and return aggregate metrics."""
    conv_flags = []
    iters_list = []
    err_list = []
    f_list = []
    g_list = []

    for r in range(R):
        opts_noisy = BFGSOptions(**{
            **base_opts.__dict__,
            "noise_sigma": float(sigma),
            "noise_seed": seed_base + 100 * sigma_index + r,
            "noise_avg": int(noise_avg),
        })

        best_sigma = multi_start_bfgs(
            target_ratio=target_ratio,
            lb=lb,
            ub=ub,
            n_starts=n_starts_per_rep,
            seed=42,
            opts=opts_noisy,
            repair_radius=repair_radius,
        )

        conv_flags.append(bool(best_sigma["converged"]))
        iters_list.append(int(best_sigma["iters"]))
        err_list.append(float(best_sigma["err_int"]))
        f_list.append(float(best_sigma["f_best"]))
        g_list.append(float(best_sigma["history_grad_norm"][-1]))

    p = float(np.mean(conv_flags))
    ci_lo, ci_hi = wilson_ci_95(p, R)

    return {
        "sigma": float(sigma),
        "conv_rate": p,
        "conv_ci_lo": ci_lo,
        "conv_ci_hi": ci_hi,
        "med_iters": float(np.median(iters_list)),
        "med_err_int": float(np.median(err_list)),
        "med_f_best": float(np.median(f_list)),
        "med_grad_norm": float(np.median(g_list)),
```

```
}

def main() -> None:
    lb = np.array([12, 12, 12, 12], dtype=float)
    ub = np.array([60, 60, 60, 60], dtype=float)
    target_ratio = 0.14427932477276006

    plot_dir = "PLOT"
    os.makedirs(plot_dir, exist_ok=True)

    # -----
    # Baseline (noise-free)
    # -----
    opts = BFGSOptions(
        max_iter=300,
        tol_grad=1e-8,
        tol_step=1e-12,
        fd_eps=1e-6,
        c1=1e-4,
        backtrack_beta=0.5,
        min_step=1e-12,
        verbose=False,
        noise_sigma=0.0,
        noise_seed=0,
        noise_avg=1,
    )

    best = multi_start_bfgs(
        target_ratio=target_ratio,
        lb=lb,
        ub=ub,
        n_starts=5000,
        seed=42,
        opts=opts,
        repair_radius=4,
    )

    x_cont = best["x_best"]
    x_int0 = best.get("x_int0", None)
    x_int = best["x_int"]
    f_best = float(best["f_best"])

    print("\n=== TASK 4 - BFGS (continuous relaxation) RESULT ===")
    print(f"Target ratio: {target_ratio:.15f}")
    print(f"Best continuous x*: {np.array2string(x_cont, precision=6, floatmode='fixed')}")
    print(f"Continuous objective: {f_best:.6e} (squared error)")
```

```
print("\n---After rounding/repair to integers (selection by integer error)---")
if x_int0 is not None:
    print(f"Rounded (pre-repair): {x_int0.tolist()}")
print(f"Integer (post-repair): {x_int.tolist()}")
print(f"Absolute error (int): {best['err_int']:.6e}")

print("\nExtra diagnostics:")
print(f"Best run start index: {best['start_index']}")
print(f"Iterations: {best['iters']}")
print(f"Final || grad ||: {best['history_grad_norm'][-1]:.6e}")
print(f"Converged flag: {best['converged']}")

# Baseline convergence plots (cleaner)
hist_f = best["history_f"]
hist_g = best["history_grad_norm"]

plt.figure()
plt.plot(hist_f, marker="o", linewidth=1.8)
plt.xlabel("Iteration")
plt.ylabel(r"$f(x)=(r(x)-r^*)^2$")
plt.title("BFGS Convergence (Objective)")
plt.grid(True, which="both", alpha=0.35)
plt.tight_layout()
plt.savefig(os.path.join(plot_dir, "task4_bfgs_convergence_objective.png"), dpi=220)

plt.figure()
plt.semilogy(hist_g, marker="o", linewidth=1.8)
plt.xlabel("Iteration")
plt.ylabel(r"$\|\nabla f(x)\|$")
plt.title("BFGS Convergence (Gradient Norm)")
plt.grid(True, which="both", alpha=0.35)
plt.tight_layout()
plt.savefig(os.path.join(plot_dir, "task4_bfgs_convergence_gradnorm.png"), dpi=220)

# -----
# Noise robustness experiment (clean & informative)
# -----
reliability_threshold = 0.8 # strict

print("\n[Noise experiment note]")
print(f"- Noise-free optimum objective:  $f^* \approx \{f_{\text{best}}\}$  .")
print(f"- Additive objective noise:  $f_{\text{noisy}}(x) = f(x) + \sigma \cdot N(0,1)$  .")
print(f"- If  $\sigma \gg f(x)$  near the optimum ( $\sim 1e-20$ ), then finite-difference gradients")
print(f"and Armijo line-search are dominated by noise, so BFGS loses reliability.\n")

# Phase 1: coarse sweep
R = 10
n_starts_per_rep = 300
repair_radius = 4
```

```
noise_avg = 10 # stable

coarse_sigmas = [0.0, 1e-15, 3e-15, 1e-14, 3e-14, 1e-13, 3e-13, 1e-12, 1e-11, 1e-10, 1e-8, 1e-6, 1e-4]
results = []

print("Coarse sweep:")
for idx, sigma in enumerate(coarse_sigmas):
    out = run_noise_point(
        sigma=sigma,
        base_opts=opts,
        target_ratio=target_ratio,
        lb=lb,
        ub=ub,
        R=R,
        n_starts_per_rep=n_starts_per_rep,
        repair_radius=repair_radius,
        noise_avg=noise_avg,
        sigma_index=idx,
    )
    results.append(out)
    print(
        f"sigma={sigma:.1e}|rate={out['conv_rate']:.2f}|"
        f" (~95% CI [{out['conv_ci_lo']:.2f}, {out['conv_ci_hi']:.2f}])|"
        f"med_iters={out['med_iters']:.0f}|med_int_err={out['med_err_int']:.3e}|"
        f"med_f_best={out['med_f_best']:.1e}|med||g||={out['med_grad_norm']:.1e}"
    )

# Identify first failure on coarse grid
failure_sigma = None
for out in results:
    if out["conv_rate"] < reliability_threshold and out["sigma"] > 0.0:
        failure_sigma = out["sigma"]
        break

# Phase 2: refine around threshold (if found)
# We refine in [sigma_prev, sigma_fail] using log-spaced points.
if failure_sigma is not None:
    # find previous sigma before failure
    prev_sigma = None
    for out in results:
        if out["sigma"] < failure_sigma and out["sigma"] > 0.0:
            prev_sigma = out["sigma"]
    if prev_sigma is not None:
        refine_sigmas = np.geomspace(prev_sigma, failure_sigma, num=7)
        print("\nRefinement sweep around the transition:")
        for j, sigma in enumerate(refine_sigmas):
            out = run_noise_point(
                sigma=float(sigma),
```

```
        base_opts=opts,
        target_ratio=target_ratio,
        lb=lb,
        ub=ub,
        R=R,
        n_starts_per_rep=n_starts_per_rep,
        repair_radius=repair_radius,
        noise_avg=noise_avg,
        sigma_index=10_000 + j,
    )
    results.append(out)
    print(
        f"sigma={sigma:.2e}|rate={out['conv_rate']:.2f}"
        f" (~95% CI [{out['conv_ci_lo']:.2f},{out['conv_ci_hi']:.2f}])"
    )

# Sort results by sigma
results = sorted(results, key=lambda d: d["sigma"])

# Recompute failure_sigma as first sigma where conv_rate < threshold
failure_sigma = None
for out in results:
    if out["sigma"] > 0.0 and out["conv_rate"] < reliability_threshold:
        failure_sigma = out["sigma"]
        break

# Console conclusion
print("\n==Noise robustness conclusion==")
if failure_sigma is None:
    print(
        "BFGS remained reliable over the tested noise range"
        f"(convergence rate never dropped below {reliability_threshold:.1f})."
    )
else:
    print(
        f"BFGS becomes unreliable at approximately noise sigma >={failure_sigma:.2e}"
        f"(convergence rate < {reliability_threshold:.1f})."
    )

# Prepare arrays for plots
sig = np.array([d["sigma"] for d in results], dtype=float)
rate = np.array([d["conv_rate"] for d in results], dtype=float)
rate_lo = np.array([d["conv_ci_lo"] for d in results], dtype=float)
rate_hi = np.array([d["conv_ci_hi"] for d in results], dtype=float)

med_iters = np.array([d["med_iters"] for d in results], dtype=float)
med_err = np.array([d["med_err_int"] for d in results], dtype=float)
med_f = np.array([d["med_f_best"] for d in results], dtype=float)
med_g = np.array([d["med_grad_norm"] for d in results], dtype=float)
```

```
# Plot 1: convergence rate with confidence interval
plt.figure()
x = sig + 1e-30
plt.semilogx(x, rate, marker="o", linewidth=1.8, label="Convergence_rate")
plt.fill_between(x, rate_lo, rate_hi, alpha=0.20, label="~95% CI (Wilson)")
plt.axhline(reliability_threshold, linestyle="--", linewidth=1.5, label=f"Threshold={
    reliability_threshold:.1f}")
if failure_sigma is not None:
    plt.axvline(failure_sigma, linestyle="--", linewidth=1.5, label=f"Estimated failure
        ≈{failure_sigma:.1e}")
plt.ylim(-0.05, 1.05)
plt.xlabel("Noise_sigma (additive on objective)")
plt.ylabel("Convergence_rate over R_runs")
plt.title("BFGS Robustness: Convergence Rate vs Objective Noise")
plt.grid(True, which="both", alpha=0.35)
plt.legend()
plt.tight_layout()
plt.savefig(os.path.join(plot_dir, "task4_bfgs_noise_convergence_rate.png"), dpi=220)

# Plot 2: median integer error
plt.figure()
plt.semilogx(x, med_err, marker="o", linewidth=1.8)
plt.xlabel("Noise_sigma (additive on objective)")
plt.ylabel("Median |ratio_int - target| after repair")
plt.title("BFGS Robustness: Median Integer Error vs Objective Noise")
plt.grid(True, which="both", alpha=0.35)
plt.tight_layout()
plt.savefig(os.path.join(plot_dir, "task4_bfgs_noise_median_integer_error.png"), dpi
    =220)

# Plot 3: median iterations
plt.figure()
plt.semilogx(x, med_iters, marker="o", linewidth=1.8)
plt.xlabel("Noise_sigma (additive on objective)")
plt.ylabel("Median iterations (best run)")
plt.title("BFGS Robustness: Median Iterations vs Objective Noise")
plt.grid(True, which="both", alpha=0.35)
plt.tight_layout()
plt.savefig(os.path.join(plot_dir, "task4_bfgs_noise_median_iterations.png"), dpi=220)

# Plot 4 (NEW): median f_best (continuous objective) -- tells the real story
plt.figure()
plt.semilogx(x, med_f, marker="o", linewidth=1.8)
plt.xlabel("Noise_sigma (additive on objective)")
plt.ylabel("Median best continuous objective f_best")
plt.title("BFGS Robustness: Median Best Objective vs Objective Noise")
plt.grid(True, which="both", alpha=0.35)
plt.tight_layout()
```

```
plt.savefig(os.path.join(plot_dir, "task4_bfgs_noise_median_fbest.png"), dpi=220)

# Plot 5 (NEW): median grad norm -- also very illustrative
plt.figure()
plt.semilogx(x, med_g, marker="o", linewidth=1.8)
plt.xlabel("Noise $\sigma$ (additive on objective)")
plt.ylabel(r"Median $\sigma\|\nabla f\|$  at termination (best run)")
plt.title("BFGS Robustness: Median Final Gradient Norm vs Objective Noise")
plt.grid(True, which="both", alpha=0.35)
plt.tight_layout()
plt.savefig(os.path.join(plot_dir, "task4_bfgs_noise_median_gradnorm.png"), dpi=220)

# Export CSV for report
csv_path = os.path.join(plot_dir, "noise_summary.csv")
with open(csv_path, "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["sigma", "conv_rate", "conv_ci_lo", "conv_ci_hi", "med_iters", "med_err_int", "med_f_best", "med_grad_norm"])
    for d in results:
        w.writerow([d["sigma"], d["conv_rate"], d["conv_ci_lo"], d["conv_ci_hi"], d["med_iters"], d["med_err_int"], d["med_f_best"], d["med_grad_norm"]])

if __name__ == "__main__":
    main()
```

A.2.2 Gear Train BFGS (gear_train_bfgs.py)

Listing 4: Gear Train BFGS Class

```
from __future__ import annotations

from dataclasses import dataclass
from typing import Callable, Dict, List, Tuple
import numpy as np

@dataclass
class BFGSOptions:
    max_iter: int = 300
    tol_grad: float = 1e-8
    tol_step: float = 1e-10
    fd_eps: float = 1e-6 # finite difference step
    c1: float = 1e-4 # Armijo parameter
    backtrack_beta: float = 0.5
    min_step: float = 1e-12
    verbose: bool = False

    # Noise injection (Task 4 robustness experiment)
    noise_sigma: float = 0.0 # std-dev of additive Gaussian noise on objective
```

```
    evaluations
noise_seed: int = 0          # RNG seed to make noisy runs reproducible
noise_avg: int = 1          # number of noisy samples averaged per evaluation

def gear_ratio(x: np.ndarray) -> float:
    """Gear_train_ratio: ratio = (N1*N3)/(N2*N4)."""
    x = np.asarray(x, dtype=float)
    return (x[0] * x[2]) / (x[1] * x[3])

def continuous_objective(x: np.ndarray, target_ratio: float) -> float:
    """Smooth objective for BFGS: f(x) = (ratio(x) - target_ratio)^2."""
    r = gear_ratio(x)
    e = r - target_ratio
    return float(e * e)

def make_noisy_objective(
    f_clean: Callable[[np.ndarray], float],
    sigma: float,
    seed: int,
    n_avg: int = 1,
) -> Callable[[np.ndarray], float]:
    """Return a callable that adds controlled Gaussian noise to objective evaluations.

    f_noisy(x) = mean_{j=1..n_avg}(f_clean(x) + sigma * N(0,1))

    Notes:
    - For sigma=0, the function reduces to f_clean.
    - A fixed seed makes the whole optimisation run reproducible.
    - Averaging (n_avg>1) reduces noise but increases evaluation cost.
    """
    if sigma <= 0.0:
        return f_clean

    rng = np.random.default_rng(seed)
    n_avg = max(1, int(n_avg))

    def f(x: np.ndarray) -> float:
        acc = 0.0
        for _ in range(n_avg):
            acc += float(f_clean(x) + sigma * rng.normal())
        return acc / n_avg

    return f

def project_box(x: np.ndarray, lb: np.ndarray, ub: np.ndarray) -> np.ndarray:
```

```
"""Projection onto box constraints [lb, ub]."""
return np.minimum(np.maximum(x, lb), ub)

def round_and_clip(x: np.ndarray, lb: np.ndarray, ub: np.ndarray) -> np.ndarray:
    """Round to nearest integer then clip into bounds."""
    xr = np rint(x).astype(int)
    xr = np.minimum(np.maximum(xr, lb.astype(int)), ub.astype(int))
    return xr

def integer_error(x_int: np.ndarray, target_ratio: float) -> float:
    """Absolute error in the true discrete problem."""
    r = gear_ratio(x_int.astype(float))
    return float(abs(r - target_ratio))

def discrete_repair_search(
    x_int0: np.ndarray,
    target_ratio: float,
    lb: np.ndarray,
    ub: np.ndarray,
    radius: int = 2,
) -> Tuple[np.ndarray, float]:
    """Local discrete search around an initial integer solution.

    Evaluates all integer combinations in the hypercube [x_int0-radius, x_int0+radius]
    (clipped to [lb, ub]) and returns the best integer design and its absolute error.
    """
    x0 = np.asarray(x_int0, dtype=int)
    lb_i = lb.astype(int)
    ub_i = ub.astype(int)

    r = int(max(0, radius))
    best_x = x0.copy()
    best_err = integer_error(best_x, target_ratio)

    ranges = []
    for i in range(len(x0)):
        lo = max(lb_i[i], x0[i] - r)
        hi = min(ub_i[i], x0[i] + r)
        ranges.append(range(lo, hi + 1))

    for n1 in ranges[0]:
        for n2 in ranges[1]:
            for n3 in ranges[2]:
                for n4 in ranges[3]:
                    cand = np.array([n1, n2, n3, n4], dtype=int)
                    err = integer_error(cand, target_ratio)
```

```
        if err < best_err:
            best_err = err
            best_x = cand

    return best_x, float(best_err)

def finite_difference_grad(f: Callable[[np.ndarray], float], x: np.ndarray, eps: float) ->
    np.ndarray:
    """Central finite difference gradient."""
    x = np.asarray(x, dtype=float)
    g = np.zeros_like(x)
    for i in range(len(x)):
        xp = x.copy()
        xm = x.copy()
        xp[i] += eps
        xm[i] -= eps
        g[i] = (f(xp) - f(xm)) / (2.0 * eps)
    return g

def armijo_backtracking(
    f: Callable[[np.ndarray], float],
    x: np.ndarray,
    p: np.ndarray,
    fx: float,
    g: np.ndarray,
    lb: np.ndarray,
    ub: np.ndarray,
    c1: float,
    beta: float,
    min_step: float,
) -> Tuple[float, np.ndarray, float]:
    """Armijo backtracking line search on projected steps."""
    alpha = 1.0
    gTp = float(np.dot(g, p))

    # If direction is not a descent direction, fall back to steepest descent
    if gTp >= 0.0:
        p = -g
        gTp = float(np.dot(g, p))

    while alpha >= min_step:
        x_new = project_box(x + alpha * p, lb, ub)
        f_new = f(x_new)
        if f_new <= fx + c1 * alpha * gTp:
            return alpha, x_new, f_new
        alpha *= beta
```

```
x_new = project_box(x + min_step * p, lb, ub)
return min_step, x_new, f(x_new)

def bfgs_minimise_box_projected(
    f_raw: Callable[[np.ndarray], float],
    x0: np.ndarray,
    lb: np.ndarray,
    ub: np.ndarray,
    opts: BFGSOptions,
) -> Dict[str, object]:
    """BFGS on a box-constrained domain via projection.

    Note: With noise_sigma > 0, objective evaluations are noisy, which intentionally
    violates deterministic assumptions and can cause loss of convergence.
    """
    x = project_box(np.asarray(x0, dtype=float), lb, ub)

    f_clean = lambda z: f_raw(project_box(z, lb, ub))
    f = make_noisy_objective(
        f_clean,
        sigma=opts.noise_sigma,
        seed=opts.noise_seed,
        n_avg=opts.noise_avg,
    )

    n = len(x)
    H = np.eye(n)

    fx = f(x)
    g = finite_difference_grad(f, x, opts.fd_eps)

    history_f: List[float] = [fx]
    history_grad_norm: List[float] = [float(np.linalg.norm(g))]
    history_alpha: List[float] = []

    if opts.verbose:
        print(f"[BFGS] iter=0 f={fx:.6e} ||g||={history_grad_norm[-1]:.6e} x={x}")

    for k in range(1, opts.max_iter + 1):
        grad_norm = float(np.linalg.norm(g))
        if grad_norm < opts.tol_grad:
            break

        p = -H @ g

        alpha, x_new, f_new = armijo_backtracking(
            f=f,
            x=x,
```

```
p=p,
fx=fx,
g=g,
lb=lb,
ub=ub,
c1=opts.c1,
beta=opts.backtrack_beta,
min_step=opts.min_step,
)

s = x_new - x
if float(np.linalg.norm(s)) < opts.tol_step:
    x, fx = x_new, f_new
    history_f.append(fx)
    history_grad_norm.append(float(np.linalg.norm(g)))
    history_alpha.append(alpha)
    break

g_new = finite_difference_grad(f, x_new, opts.fd_eps)
y = g_new - g

ys = float(np.dot(y, s))
if ys > 1e-12:
    rho = 1.0 / ys
    I = np.eye(n)
    V = I - rho * np.outer(s, y)
    H = V @ H @ V.T + rho * np.outer(s, s)
else:
    H = np.eye(n)

x, fx, g = x_new, f_new, g_new

history_f.append(fx)
history_grad_norm.append(float(np.linalg.norm(g)))
history_alpha.append(alpha)

if opts.verbose and (k % 10 == 0 or k == 1):
    print(
        f"[BFGS]_iter={k}_f={fx:.6e}_||g||={history_grad_norm[-1]:.6e}_\n"
        f"alpha={alpha:.2e}_x={x}"
    )

# Conservative convergence flag under noise:
converged = (history_grad_norm[-1] < opts.tol_grad) or (
    len(history_f) >= 2 and float(abs(history_f[-1] - history_f[-2])) < 1e-30
)

return {
    "x_best": x,
```

```
        "f_best": fx,
        "history_f": np.array(history_f, dtype=float),
        "history_grad_norm": np.array(history_grad_norm, dtype=float),
        "history_alpha": np.array(history_alpha, dtype=float),
        "iters": len(history_f) - 1,
        "converged": bool(converged),
    }

}

def multi_start_bfgs(
    target_ratio: float,
    lb: np.ndarray,
    ub: np.ndarray,
    n_starts: int = 500,
    seed: int = 42,
    opts: BFGSOptions | None = None,
    repair_radius: int = 2,
) -> Dict[str, object]:
    """Multi-start BFGS.

    IMPORTANT:
    We SELECT the best candidate primarily by integer error (after rounding+repair),
    and use continuous objective as a tie-breaker.
    """
    if opts is None:
        opts = BFGSOptions()

    rng = np.random.default_rng(seed)
    best = None

    def f_raw(x: np.ndarray) -> float:
        return continuous_objective(x, target_ratio=target_ratio)

    for i in range(n_starts):
        x0 = rng.uniform(lb, ub)
        out = bfgs_minimise_box_projected(
            f_raw=f_raw,
            x0=x0,
            lb=lb,
            ub=ub,
            opts=opts,
        )

        x_cont = out["x_best"]
        f_cont = float(out["f_best"])

        x_int0 = round_and_clip(x_cont, lb, ub)
        x_int, err_int = discrete_repair_search(
            x_int0=x_int0,
```

```
        target_ratio=target_ratio,
        lb=lb,
        ub=ub,
        radius=repair_radius,
    )

    record = {
        **out,
        "start_index": i,
        "x0": x0,
        "x_int0": x_int0,
        "x_int": x_int,
        "err_int": float(err_int),
        "ratio_cont": gear_ratio(x_cont),
        "ratio_int": gear_ratio(x_int.astype(float)),
    }

    if best is None:
        best = record
        continue

    # Primary criterion: best integer error
    if record["err_int"] < float(best["err_int"]):
        best = record
        continue

    # Tie-breaker: best continuous objective
    if record["err_int"] == float(best["err_int"]) and f_cont < float(best["f_best"]):
        best = record

    assert best is not None
    return best
```

A.3 Two Bar Plane Truss problem

A.3.1 Custom BFGS script for Two Truss problem(custom_TBT_BFGS.py)

Listing 5: Main execution script for Task 4 protocols

```
"""
Subject: Custom BFGS Implementation for Two Bar Truss
@author: Cl  mence - Philom  ne HINOT + Claude AI
@date: 17/01/2026
@Description: This module implements BFGS optimization from scratch.
"""

# ***** IMPORTATION *****
```

```
from __future__ import annotations
from dataclasses import dataclass
from typing import Callable, Dict, List, Tuple
import numpy as np

# ***** CUSTOM BFGS *****

@dataclass
class BFGSOptions:
    """Configuration options for BFGS optimizer"""
    max_iter: int = 300          # Maximum iterations
    tol_grad: float = 1e-8       # Gradient norm tolerance for convergence
    tol_step: float = 1e-12      # Step size tolerance
    fd_eps: float = 1e-6         # Finite difference step size
    c1: float = 1e-4             # Armijo condition parameter
    backtrack_beta: float = 0.5  # Backtracking reduction factor
    min_step: float = 1e-12      # Minimum step size
    verbose: bool = False       # Print iteration details

def project_box(x: np.ndarray, lb: np.ndarray, ub: np.ndarray) -> np.ndarray:
    """
    Project point x onto box constraints [lb, ub].

    For each component: x_i = max(lb_i, min(x_i, ub_i))
    """
    return np.minimum(np.maximum(x, lb), ub)

def finite_difference_grad(
    f: Callable[[np.ndarray], float],
    x: np.ndarray,
    eps: float
) -> np.ndarray:
    """
    Compute gradient using central finite differences.

    For each dimension i:
     $\partial f / \partial x_i \approx [f(x + \epsilon \cdot e_i) - f(x - \epsilon \cdot e_i)] / (2\epsilon)$ 

    Cost: 2n function evaluations (where n = len(x))

    Args:
        f: Objective function
        x: Current point
        eps: Finite difference step size

    Returns:
        Gradient vector  $\nabla f(x)$ 
    """
```

```
"""
    x = np.asarray(x, dtype=float)
    n = len(x)
    g = np.zeros(n, dtype=float)

    for i in range(n):
        # Perturb in positive direction
        x_plus = x.copy()
        x_plus[i] += eps
        f_plus = f(x_plus)

        # Perturb in negative direction
        x_minus = x.copy()
        x_minus[i] -= eps
        f_minus = f(x_minus)

        # Central difference
        g[i] = (f_plus - f_minus) / (2.0 * eps)

    return g

def armijo_backtracking(
    f: Callable[[np.ndarray], float],
    x: np.ndarray,
    p: np.ndarray,
    fx: float,
    g: np.ndarray,
    lb: np.ndarray,
    ub: np.ndarray,
    c1: float,
    beta: float,
    min_step: float,
) -> Tuple[float, np.ndarray, float]:
    """
    Armijo backtracking line search with box constraint projection.

    Finds step size  $\alpha$  such that the Armijo condition is satisfied:
     $f(x + \alpha \cdot p) \leq f(x) + c1 \cdot \alpha \cdot \nabla f(x)^T p$  (sufficient decrease)

    Algorithm:
    1. Start with  $\alpha = 1$ 
    2. While Armijo condition not satisfied:
        Reduce  $\alpha \leftarrow \beta \cdot \alpha$ 
        Project  $x + \alpha \cdot p$  onto  $[lb, ub]$ 
    3. Return  $\alpha$ , new point, new function value

    Args:
    f: Objective function
```

```
#####x: Current point
#####p: Search direction
#####fx: f(x), function value at current point
#####g: ∇f(x), gradient at current point
#####lb, ub: Box constraints
#####c1: Armijo parameter (typically 1e-4)
#####beta: Backtracking factor (typically 0.5)
#####min_step: Minimum allowed step size

#####Returns:
#####(alpha, x_new, f_new): Step size, new point, new function value
#####"""
    alpha = 1.0
    gTp = float(np.dot(g, p))

    # Safety check: if p is not a descent direction, use steepest descent
    if gTp >= 0.0:
        p = -g
        gTp = float(np.dot(g, p))

    # Backtracking loop
    while alpha >= min_step:
        # Proposed step with projection
        x_new = project_box(x + alpha * p, lb, ub)
        f_new = f(x_new)

        # Check Armijo condition
        if f_new <= fx + c1 * alpha * gTp:
            return alpha, x_new, f_new

        # Reduce step size
        alpha *= beta

    # If no acceptable step found, take minimum step
    x_new = project_box(x + min_step * p, lb, ub)
    return min_step, x_new, f(x_new)

def bfgs_update(
    H: np.ndarray,
    s: np.ndarray,
    y: np.ndarray
) -> np.ndarray:
    """
    #####BFGS update formula for inverse Hessian approximation.

    #####Standard BFGS formula:
    #####ρ = 1 / (yTs)
    #####V = I - ρ · s · yT
```

```
#####  $H_{k+1} = V \cdot H_k \cdot V^T + \rho \cdot s \cdot s^T$ 

#####Where:
#####  $s = x_{k+1} - x_k(\text{step})$ 
#####  $y = \nabla f_{k+1} - \nabla f_k(\text{gradient difference})$ 
#####  $H \approx [\nabla^2 f]$  #####  $(\text{inverse Hessian approximation})$ 

#####Curvature condition:  $y^T s > 0$  must hold for positive definiteness.
#####If violated, reset  $H$  to identity matrix.

#####Args:
#####H: Current inverse Hessian approximation
#####s: Step vector
#####y: Gradient difference

#####Returns:
#####Updated inverse Hessian approximation
#####"""
    n = len(s)
    ys = float(np.dot(y, s))

    # Check curvature condition
    if ys > 1e-12:
        # BFGS update
        rho = 1.0 / ys
        I = np.eye(n)
        V = I - rho * np.outer(s, y)
        H_new = V @ H @ V.T + rho * np.outer(s, s)
        return H_new
    else:
        # Curvature condition violated - reset to identity
        return np.eye(n)

def bfgs_minimize(
    f: Callable[[np.ndarray], float],
    x0: np.ndarray,
    lb: np.ndarray,
    ub: np.ndarray,
    opts: BFGSOptions | None = None,
) -> Dict[str, object]:
    """
    #####BFGS minimization with box constraints (via projection).

    #####Algorithm:
    #####1. Initialize:  $H = I$  (identity matrix),  $x = \text{project}(x_0)$ 
    #####2. For  $k = 0, 1, 2, \dots$ 
    #####a. Compute gradient:  $g = \nabla f(x)$ 
    #####b. Check convergence:  $\|g\| < \text{tol\_grad}$ 
```

```
#####c.Search_direction: p=-H·g
#####d.Line_search: find α satisfying Armijo condition
#####e.Update: x←x+α·p, project onto bounds
#####f.BFGS_update: H←BFGS(H, s, y)
#####3.Return solution

#####Args:
#####f: Objective function to minimize
#####x0: Initial guess
#####lb: Lower bounds
#####ub: Upper bounds
#####opts: BFGS options (use defaults if None)

#####Returns:
#####Dictionary containing:
#####- x_best: Optimal solution found
#####- f_best: Objective value at solution
#####- history_f: Function values at each iteration
#####- history_grad_norm: Gradient norms at each iteration
#####- history_alpha: Step sizes at each iteration
#####- iters: Number of iterations performed
#####- converged: Boolean convergence flag
#####"""
    if opts is None:
        opts = BFGSOptions()

    # Initialize
    x = project_box(np.asarray(x0, dtype=float), lb, ub)
    n = len(x)
    H = np.eye(n) # Initial inverse Hessian approximation

    # Evaluate initial point
    fx = f(x)
    g = finite_difference_grad(f, x, opts.fd_eps)

    # History tracking
    history_f: List[float] = [fx]
    history_grad_norm: List[float] = [float(np.linalg.norm(g))]
    history_alpha: List[float] = []

    if opts.verbose:
        print(f"[BFGS] Iteration 0:")
        print(f"    f(x)={fx:.10e}")
        print(f"    ||∇f||={history_grad_norm[-1]:.6e}")
        print(f"    x={x}")

    # Main BFGS loop
    for k in range(1, opts.max_iter + 1):
        grad_norm = float(np.linalg.norm(g))
```

```
# Convergence check: gradient norm
if grad_norm < opts.tol_grad:
    if opts.verbose:
        print(f"\n[BFGS] Converged: || $\nabla f$ || = {grad_norm:.6e} < {opts.tol_grad:.6e}")
    break

# Search direction:  $p = -H \cdot g$  (quasi-Newton direction)
p = -H @ g

# Line search with Armijo backtracking
alpha, x_new, f_new = armijo_backtracking(
    f=f,
    x=x,
    p=p,
    fx=fx,
    g=g,
    lb=lb,
    ub=ub,
    c1=opts.c1,
    beta=opts.backtrack_beta,
    min_step=opts.min_step,
)

# Compute step
s = x_new - x

# Convergence check: step size
if float(np.linalg.norm(s)) < opts.tol_step:
    x, fx = x_new, f_new
    history_f.append(fx)
    history_grad_norm.append(float(np.linalg.norm(g)))
    history_alpha.append(alpha)
    if opts.verbose:
        print(f"\n[BFGS] Converged: ||step|| = {np.linalg.norm(s):.6e} < {opts.tol_step:.6e}")
    break

# Gradient at new point
g_new = finite_difference_grad(f, x_new, opts.fd_eps)

# Gradient difference
y = g_new - g

# BFGS update of inverse Hessian approximation
H = bfgs_update(H, s, y)

# Update state
x, fx, g = x_new, f_new, g_new
```

```
# Record history
history_f.append(fx)
history_grad_norm.append(float(np.linalg.norm(g)))
history_alpha.append(alpha)

# Verbose output
if opts.verbose and (k % 10 == 0 or k == 1):
    print(f"\n[BFGS] Iteration {k}:")
    print(f"    f(x) = {fx:.10e}")
    print(f"    ||∇f|| = {history_grad_norm[-1]:.6e}")
    print(f"    α = {alpha:.6e}")
    print(f"    x = {x}")

# Check gradient convergence
grad_converged = history_grad_norm[-1] < opts.tol_grad

# Check step convergence (practical convergence)
step_converged = False
if len(history_alpha) > 0 and history_alpha[-1] < opts.min_step * 10:
    step_converged = True

# Combined: converged if EITHER criterion is met
converged = grad_converged or step_converged

# Determine termination reason
if grad_converged:
    termination = "Gradient norm below tolerance"
elif step_converged:
    termination = "Step size below threshold (practical convergence)"
else:
    termination = "Maximum iterations reached"

return {
    "x_best": x,
    "f_best": fx,
    "history_f": np.array(history_f, dtype=float),
    "history_grad_norm": np.array(history_grad_norm, dtype=float),
    "history_alpha": np.array(history_alpha, dtype=float),
    "iters": len(history_f) - 1,
    "converged": bool(converged),
    "grad_converged": bool(grad_converged),
    "step_converged": bool(step_converged),
    "termination": termination,
}
```

A.3.2 Two Truss problem script adapted for BFGS (two_bar_plane_truss_problem.py)

Listing 6: Main execution script for Task 4 protocols

```
"""
Subject: Two_Bar_Plane_Truss_Problem adapted for BFGS optimisation
Source: https://www.sciencedirect.com/science/article/pii/S0898122111010406
@author: Clemence-Philomene HINOT & Claude AI
@date: 12/01/2026
"""

# ***** IMPORTATION *****

import math

# ***** CLASS DEFINITION *****

class Two_Bar_Plane_Truss_Problem():

    # ----- INITIALIZATION -----

    def __init__(self):
        # constants
        self.rho = 7833          # unit: kg/m^3
        self.h = 2.54            # unit: m
        self.P = 44482.2         # unit: N
        self.E = 2.07e11         # unit: Pa
        self.sigma0 = 1.38e8     # unit: Pa
        self.A_min = 6.4516e-4   # unit: m^2

        # variables
        self.pos = 0.0
        self.area = 0.0

        # physical boundaries
        self.pos_min = 0.1 * self.h
        self.pos_max = 2.25 * self.h
        self.area_min = 0.5 * self.A_min
        self.area_max = 2.5 * self.A_min

    # ----- CONSTITUTIVE EQUATIONS -----

    # Weight objective
    def f1(self):
        x1 = self.pos/self.h # need to use normalised x1 in this function
        return 2 * self.rho * self.h * self.area * math.sqrt(1 + pow(x1, 2))

    # Displacement objective
    def f2(self):
        x1 = self.pos/self.h
        return (self.P * pow(self.h, 3) * pow((1 + pow(x1, 2)), 1.5)) / ( math.sqrt(1 + pow(
```

```
        x1, 4)) * 2 * math.sqrt(2) * self.E * x1 * self.area)

# ----- CONSTRAINT EQUATIONS -----

# Stress constraint (tension)
def g1(self, normalised=True):
    x1 = self.pos/self.h
    stress = (self.P * (1 + x1) * math.sqrt(1 + pow(x1, 2)) / (2 * math.sqrt(2) * x1 *
        self.area))

    if normalised:
        return (stress / self.sigma0) - 1.0 # normalised result
    else:
        return stress - self.sigma0 # real result in Pa

# Stress constraint (compression)
def g2(self, normalised=True):
    x1 = self.pos/self.h
    stress = (self.P * (1 - x1) * math.sqrt(1 + pow(x1, 2)) / (2 * math.sqrt(2) * x1 *
        self.area))

    if normalised:
        return (stress / self.sigma0) - 1.0 # normalised result
    else:
        return stress - self.sigma0 # real result in Pa

# ----- EVALUATION FUNCTION -----

def evaluate(self, x):
    """
    Evaluate objectives and constraints

    param x: [x1_normalized, x2_normalized]
    return: (sum_value, [f1, f2], [g1, g2])
    """

    # physical position
    self.pos = x[0] * self.h

    # physical area
    self.area = x[1] * self.A_min

    # weight
    f1 = self.f1()

    # movement
    f2 = self.f2()
```

```
# constraints
g1 = self.g1()
g2 = self.g2()

# penalty to apply if the constraint g > 0
penalty1 = max(0, g1)**2
penalty2 = max(0, g2)**2

# weight for the constraint functions
weightConstraint = 10000.0

# weight for f1 and f2
# they have the same
weightFunction = 0.5

# weighted sum
sum = weightFunction * f1 + weightFunction * f2 + weightConstraint * (penalty1 +
    penalty2)

# results
return sum, [f1, f2], [g1, g2]
```

A.3.3 Two Truss without noise main script(main_TBT_BFGS.py)

Listing 7: Main execution script for Task 4 protocols

```
"""
Subject:MainScript
@author:Clémence-Philomène HINOT+Claude AI
@date:15/01/2026
@Description:Custom BFGS implementation for the Two Bar Truss problem.
Adapted from the Gear Train BFGS approach.
"""

# ***** IMPORTATION *****

# mathematics
import numpy as np

# visualisation
import matplotlib.pyplot as plt

# store data
import os

# custom BFGS algorithm
from custom_TBT_BFGS import bfgs_minimize, BFGSOptions

# Two Bar Truss problem
```

```
from two_bar_plane_truss_problem import Two_Bar_Plane_Truss_Problem

# ***** TWO BAR TRUSS PROBLEM *****

# create an instance of the truss problem
truss_problem = Two_Bar_Plane_Truss_Problem()

# define objective function and keep only the sum value
def objective(x):
    # wrapped
    sum_value, objectives, constraints = truss_problem.evaluate(x)
    return sum_value

# problem bounds
lb = np.array([0.1, 0.5])    # lower bounds: [x1_min, x2_min]
ub = np.array([2.25, 2.5])  # upper bounds: [x1_max, x2_max]

# ***** OPTIMISATION *****

# initial guess
x0 = np.array([1.0, 1.5])

# BFGS options
opts = BFGSOptions(
    max_iter=50,          # Maximum iterations
    tol_grad=1e-5,        # Gradient norm tolerance
    tol_step=1e-12,       # Step size tolerance
    fd_eps=1e-6,          # Finite difference step
    c1=1e-4,              # Armijo parameter
    backtrack_beta=0.5,   # Backtracking factor
    min_step=1e-12,       # Minimum step size
    verbose=False         # Set to True to see iteration details
)

# BFGS optimisation
result = bfgs_minimize(
    f = objective,        # function to minimise
    x0=x0,                # initial guess
    lb=lb,                # lower bound
    ub=ub,                # upper bound
    opts=opts             # options
)

# evaluate final solution
final_sum, final_obj, final_con = truss_problem.evaluate(result["x_best"])

# calculate physical values
position = result["x_best"][0] * truss_problem.h
area = result["x_best"][1] * truss_problem.A_min
```

```
# check constraint satisfaction
constraints_satisfied = (final_con[0] <= 0) and (final_con[1] <= 0)
in_bounds = ((lb[0] <= result["x_best"][0] <= ub[0]) and (lb[1] <= result["x_best"][1] <= ub
[1]))

# ***** DISPLAY RESULTS *****

print(f"\nConvergence_Status:")
print(f"    S: {result['converged']}")
print(f"    Iterations: {result['iters']}")
print(f"    Function evaluations: {result['iters']*2*len(x0)+1}")
print(f"    (1 initial + {result['iters']} iterations * {2*len(x0)} for finite differences)")

print(f"\nOptimal_Solution_(normalized_variables):")
print(f"    x1(position_factor) = {result['x_best'][0]:.10f}")
print(f"    x2(area_factor) = {result['x_best'][1]:.10f}")
print(f"    Within bounds: {'✓YES' if in_bounds else '✗NO'}")

print(f"\nPhysical_Solution:")
print(f"    Position = {position:.8f} m")
print(f"    Area = {area:.8e} m2")

print(f"\nObjective_Function:")
print(f"    Combined objective = {result['f_best']:.12e}")
print(f"    f1(weight) = {final_obj[0]:.8f} kg")
print(f"    f2(displacement) = {final_obj[1]:.8e} m")

print(f"\nConstraints:")
print(f"    Normalized (for optimization):")
print(f"    g1 = {final_con[0]:+.8e} {'✓' if final_con[0] <= 0 else '✗'}")
print(f"    g2 = {final_con[1]:+.8e} {'✓' if final_con[1] <= 0 else '✗'}")

# Get actual constraint values using normalized=False
g1_actual = truss_problem.g1(normalised=False)
g2_actual = truss_problem.g2(normalised=False)

print(f"    Actual (Pa):")
print(f"    g1( $\sigma_{\text{tension}}$  -  $\sigma_0$ ) = {g1_actual:+.6e} Pa {'✓' if g1_actual <= 0 else '✗'}")
print(f"    g2( $\sigma_{\text{compression}}$  -  $\sigma_0$ ) = {g2_actual:+.6e} Pa {'✓' if g2_actual <= 0 else '✗'}")

print(f"\nConvergence_Metrics:")
print(f"    Final gradient norm: {result['history_grad_norm'][-1]:.8e}")
if len(result['history_alpha']) > 0:
    print(f"    Final step size  $\alpha$ : {result['history_alpha'][-1]:.8e}")

# ***** VISUALISATION *****
```

```
# Create plots directory
plot_dir = "TBT_PLOT"
os.makedirs(plot_dir, exist_ok=True)

# Generate convergence plots
print("\nGenerating convergence plots...")

# Plot 1: Objective function convergence
plt.figure(figsize=(10, 6))
plt.semilogy(result["history_f"], 'b-o', linewidth=2, markersize=5, label='Objective f(x)')
plt.xlabel('Iteration', fontsize=12)
plt.ylabel('Objective Function Value', fontsize=12)
plt.title('BFGS Convergence: Objective Function', fontsize=14, fontweight='bold')
plt.grid(True, which='both', alpha=0.3)
plt.legend(fontsize=11)
plt.tight_layout()
plt.savefig(os.path.join(plot_dir, 'bfgs_convergence_objective.png'), dpi=300)
print(f"Saved: {plot_dir}/bfgs_convergence_objective.png")

# Plot 2: Gradient norm convergence
plt.figure(figsize=(10, 6))
plt.semilogy(result["history_grad_norm"], 'r-o', linewidth=2, markersize=5, label='Gradient Norm')
plt.axhline(y=opts.tol_grad, color='g', linestyle='--', linewidth=2,
            label=f'Tolerance={opts.tol_grad:.0e}')
plt.xlabel('Iteration', fontsize=12)
plt.ylabel(r'Gradient Norm $\|\nabla f(x)\|$', fontsize=12)
plt.title('BFGS Convergence: Gradient Norm', fontsize=14, fontweight='bold')
plt.grid(True, which='both', alpha=0.3)
plt.legend(fontsize=11)
plt.tight_layout()
plt.savefig(os.path.join(plot_dir, 'bfgs_convergence_gradient.png'), dpi=300)
print(f"Saved: {plot_dir}/bfgs_convergence_gradient.png")

# Plot 3: Step sizes (line search)
if len(result["history_alpha"]) > 0:
    plt.figure(figsize=(10, 6))
    plt.semilogy(range(1, len(result["history_alpha"]) + 1),
                 result["history_alpha"], 'g-o', linewidth=2, markersize=5,
                 label='Step Size $\alpha$')
    plt.xlabel('Iteration', fontsize=12)
    plt.ylabel(r'Step Size $\alpha$ (from line search)', fontsize=12)
    plt.title('BFGS: Line Search Step Sizes', fontsize=14, fontweight='bold')
    plt.grid(True, which='both', alpha=0.3)
    plt.legend(fontsize=11)
    plt.tight_layout()
    plt.savefig(os.path.join(plot_dir, 'bfgs_step_sizes.png'), dpi=300)
    print(f"Saved: {plot_dir}/bfgs_step_sizes.png")
```

```
plt.close('all')
```

A.3.4 Two Truss with noise main script(main_TBT_BFGS_noise.py)

Listing 8: Main execution script for Task 4 protocols

```
"""
Subject: Main Script
@author: Pierre Bédune + Claude AI
@date: 15/01/2026
@Description: Custom BFGS implementation for the Two Bar Truss problem.
             Adapted from the Gear Train BFGS approach.
"""

# ***** IMPORTATION *****

# mathematics
import numpy as np

# visualisation
import matplotlib.pyplot as plt

# store data
import os

# custom BFGS algorithm
from custom_TBT_BFGS import bfgs_minimize, BFGSOptions

# Two Bar Truss problem
from two_bar_plane_truss_problem import Two_Bar_Plane_Truss_Problem

# ***** NOISE WRAPPER *****

def make_noisy_objective(f_clean, sigma, seed=0):
    """
    Build a noisy objective:
    f_noisy(x) = f_clean(x) + sigma * N(0,1)
    """
    rng = np.random.default_rng(seed)

    if sigma <= 0.0:
        return f_clean

    def f_noisy(x):
        return float(f_clean(x)) + float(sigma) * float(rng.normal())

    return f_noisy
```

```
# ***** TWO BAR TRUSS PROBLEM *****

# create an instance of the truss problem
truss_problem = Two_Bar_Plane_Truss_Problem()

# define objective function and keep only the sum value
def objective_clean(x):
    sum_value, objectives, constraints = truss_problem.evaluate(x)
    return sum_value

# problem bounds
lb = np.array([0.1, 0.5])    # lower bounds: [x1_min, x2_min]
ub = np.array([2.25, 2.5])  # upper bounds: [x1_max, x2_max]

# ***** OPTIMISATION *****

# initial guess
x0 = np.array([1.0, 1.5])

# BFGS options
opts = BFGSOptions(
    max_iter=1000,          # Maximum iterations
    tol_grad=1e-5,          # Gradient norm tolerance
    tol_step=1e-40,         # Step size tolerance
    fd_eps=1e-6,           # Finite difference step
    c1=1e-4,               # Armijo parameter
    backtrack_beta=0.5,     # Backtracking factor
    min_step=1e-12,         # Minimum step size
    verbose=False           # Set to True to see iteration details
)

# ***** VISUALISATION *****

# Create plots directory
plot_dir = "TBT_PLOT"
os.makedirs(plot_dir, exist_ok=True)

# ***** RUN WITH 3 NOISE LEVELS *****

sigmas = [0.0, 1e-4, 1e-2, 1]    # <- change ici si tu veux d'autres niveaux
base_seed = 42                   # seed de base (reproductible)

for sigma in sigmas:

    # objective = noisy(objective_clean)
    objective = make_noisy_objective(objective_clean, sigma=sigma, seed=base_seed)

    # BFGS optimisation
```

```
result = bfgs_minimize(
    f=objective,          # function to minimise
    x0=x0,                # initial guess
    lb=lb,                # lower bound
    ub=ub,                # upper bound
    opts=opts             # options
)

# evaluate final solution (sur la fonction "clean" physique, sans bruit)
final_sum, final_obj, final_con = truss_problem.evaluate(result["x_best"])

# calculate physical values
position = result["x_best"][0] * truss_problem.h
area = result["x_best"][1] * truss_problem.A_min

# check constraint satisfaction
constraints_satisfied = (final_con[0] <= 0) and (final_con[1] <= 0)
in_bounds = ((lb[0] <= result["x_best"][0] <= ub[0]) and (lb[1] <= result["x_best"][1]
    <= ub[1]))

# ***** DISPLAY RESULTS *****

print("\n" + "="*70)
print(f"RUN WITH NOISE: sigma={sigma:.2e}")
print("="*70)

print(f"\nConvergence Status:")
print(f"  S: {result['converged']}")
print(f"  Iterations: {result['iters']}")
print(f"  Function evaluations: {result['iters']*2*len(x0)+1}")
print(f"  (1 initial + {result['iters']} iterations x {2*len(x0)} for finite differences)")

print(f"\nOptimal Solution (normalized variables):")
print(f"  x1(position factor) = {result['x_best'][0]:.10f}")
print(f"  x2(area factor) = {result['x_best'][1]:.10f}")
print(f"  Within bounds: {'✓ YES' if in_bounds else '✗ NO'}")

print(f"\nPhysical Solution:")
print(f"  Position = {position:.8f} m")
print(f"  Area = {area:.8e} m2")

print(f"\nObjective Function:")
print(f"  Combined objective (noisy during optimisation) = {result['f_best']:.12e}")
print(f"  Combined objective (clean re-eval) = {final_sum:.12e}")
print(f"  f1(weight) = {final_obj[0]:.8f} kg")
print(f"  f2(displacement) = {final_obj[1]:.8e} m")

print(f"\nConstraints:")
```

```
print(f"Normalized(for optimization):")
print(f"g1={final_con[0]:+.8e}{'✓' if final_con[0] <= 0 else '×'}")
print(f"g2={final_con[1]:+.8e}{'✓' if final_con[1] <= 0 else '×'}")

# Get actual constraint values using normalized=False
g1_actual = truss_problem.g1(normalised=False)
g2_actual = truss_problem.g2(normalised=False)

print(f"Actual(Pa):")
print(f"g1(σ_tension-σ0)={g1_actual:+.6e}Pa{'✓' if g1_actual <= 0 else '×'}")
print(f"g2(σ_compression-σ0)={g2_actual:+.6e}Pa{'✓' if g2_actual <= 0 else '×'}")

print(f"\nConvergence Metrics:")
print(f"Final gradient_norm: {result['history_grad_norm'][-1]:.8e}")
if len(result['history_alpha']) > 0:
    print(f"Final step_size: {result['history_alpha'][-1]:.8e}")

# ***** VISUALISATION *****

print("\nGenerating convergence plots...")

tag = f"sigma_{sigma:.0e}".replace("+", "").replace("-", "m") # filename-safe

# Plot 1: Objective function convergence
plt.figure(figsize=(10, 6))
plt.semilogy(result["history_f"], 'b-o', linewidth=2, markersize=5, label='Objective f(x)')
plt.xlabel('Iteration', fontsize=12)
plt.ylabel('Objective Function Value', fontsize=12)
plt.title(f'BFGS Convergence: Objective Function (sigma={sigma:.0e})', fontsize=14,
          fontweight='bold')
plt.grid(True, which='both', alpha=0.3)
plt.legend(fontsize=11)
plt.tight_layout()
plt.savefig(os.path.join(plot_dir, f'bfgs_convergence_objective_{tag}.png'), dpi=300)
print(f"Saved: {plot_dir}/bfgs_convergence_objective_{tag}.png")

# Plot 2: Gradient norm convergence
plt.figure(figsize=(10, 6))
plt.semilogy(result["history_grad_norm"], 'r-o', linewidth=2, markersize=5, label='Gradient Norm')
plt.axhline(y=opts.tol_grad, color='g', linestyle='--', linewidth=2,
            label=f'Tolerance={opts.tol_grad:.0e}')
plt.xlabel('Iteration', fontsize=12)
plt.ylabel(r'Gradient Norm $\|\nabla f(x)\|$', fontsize=12)
plt.title(f'BFGS Convergence: Gradient Norm (sigma={sigma:.0e})', fontsize=14,
          fontweight='bold')
```

```
plt.grid(True, which='both', alpha=0.3)
plt.legend(fontsize=11)
plt.tight_layout()
plt.savefig(os.path.join(plot_dir, f'bfgs_convergence_gradient_{tag}.png'), dpi=300)
print(f"Saved: {plot_dir}/bfgs_convergence_gradient_{tag}.png")

# Plot 3: Step sizes (line search)
if len(result["history_alpha"]) > 0:
    plt.figure(figsize=(10, 6))
    plt.semilogy(range(1, len(result["history_alpha"]) + 1),
                 result["history_alpha"], 'g-o', linewidth=2, markersize=5,
                 label='Step Size  $\alpha$ ')
    plt.xlabel('Iteration', fontsize=12)
    plt.ylabel(r'Step Size  $\alpha$  (from line search)', fontsize=12)
    plt.title(f'BFGS: Line Search Step Sizes (sigma={sigma:.0e})', fontsize=14,
              fontweight='bold')
    plt.grid(True, which='both', alpha=0.3)
    plt.legend(fontsize=11)
    plt.tight_layout()
    plt.savefig(os.path.join(plot_dir, f'bfgs_step_sizes_{tag}.png'), dpi=300)
    print(f"Saved: {plot_dir}/bfgs_step_sizes_{tag}.png")

plt.close('all')
```

A.3.5 Test script for Two Truss problem with noise(Test_noise.py)

Listing 9: Main execution script for Task 4 protocols

```
"""
Subject: BFGS Limits Testing with Noise
@author: Pierre Bédrupe + Claude AI
@date: 17/01/2026
@Description: Systematic testing of BFGS algorithm limits with various noise levels
              using CURRENT parameters to identify breaking points
"""

# ***** IMPORTATION *****

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import os
import time
from typing import Dict, List, Tuple
import sys

sys.path.insert(0, '/mnt/project')

from custom_TBT_BFGS import bfgs_minimize, BFGSOptions
```

```
from two_bar_plane_truss_problem import Two_Bar_Plane_Truss_Problem

# ***** NOISE WRAPPER *****

def make_noisy_objective(f_clean, sigma, seed=0):
    """
    Create noisy version of objective function
    f_noisy(x) = f_clean(x) +  $\sigma$  * N(0,1)
    """
    rng = np.random.default_rng(seed)

    if sigma <= 0.0:
        return f_clean

    def f_noisy(x):
        return float(f_clean(x)) + float(sigma) * float(rng.normal())

    return f_noisy

# ***** EVALUATION FUNCTION *****

def evaluate_bfgs_performance(
    objective_clean,
    objective_noisy,
    x0: np.ndarray,
    lb: np.ndarray,
    ub: np.ndarray,
    opts: BFGSOptions,
    sigma: float,
    problem_instance,
    run_id: int
) -> Dict:
    """
    Evaluate BFGS performance for a single run

    Returns:
    Dictionary with performance metrics
    """

    # Run optimization
    start_time = time.time()
    result = bfgs_minimize(
        f=objective_noisy,
        x0=x0,
        lb=lb,
        ub=ub,
        opts=opts
    )
```

```
elapsed_time = time.time() - start_time

# Evaluate on CLEAN objective (ground truth)
final_sum_clean, final_obj, final_con = problem_instance.evaluate(result["x_best"])

# Evaluate on NOISY objective (what optimizer saw)
final_sum_noisy = objective_noisy(result["x_best"])

# Constraint satisfaction
constraints_ok = (final_con[0] <= 0) and (final_con[1] <= 0)
bounds_ok = np.all((lb <= result["x_best"]) & (result["x_best"] <= ub))

# Feasibility
feasible = constraints_ok and bounds_ok

# Convergence quality
if len(result["history_f"]) > 1:
    # Relative improvement between first and last iteration
    improvement = abs(result["history_f"][0] - result["history_f"][-1]) / (abs(result["history_f"][0]) + 1e-10)
else:
    improvement = 0.0

# Oscillation measure (variance in last 20% of iterations)
n_tail = max(1, int(0.2 * len(result["history_f"])))
tail_values = result["history_f"][-n_tail:]
oscillation = float(np.std(tail_values)) if len(tail_values) > 1 else 0.0

return {
    'run_id': run_id,
    'sigma': sigma,
    'converged': result['converged'],
    'grad_converged': result['grad_converged'],
    'step_converged': result['step_converged'],
    'termination': result['termination'],
    'iterations': result['iters'],
    'time_sec': elapsed_time,
    'f_final_noisy': result['f_best'],
    'f_final_clean': final_sum_clean,
    'f_initial': result["history_f"][0],
    'improvement_pct': improvement * 100,
    'oscillation': oscillation,
    'grad_norm_final': result['history_grad_norm'][-1],
    'step_size_final': result['history_alpha'][-1] if len(result['history_alpha']) > 0
        else np.nan,
    'x1_final': result['x_best'][0],
    'x2_final': result['x_best'][1],
    'f1_weight': final_obj[0],
    'f2_displacement': final_obj[1],
```

```
'g1_constraint': final_con[0],
'g2_constraint': final_con[1],
'constraints_satisfied': constraints_ok,
'bounds_satisfied': bounds_ok,
'feasible': feasible,
'history_f': result["history_f"],
'history_grad': result["history_grad_norm"],
'history_alpha': result["history_alpha"]
}

# ***** MAIN TEST FUNCTION *****

def test_bfgs_limits(
    noise_levels: List[float],
    n_runs_per_noise: int = 10,
    output_dir: str = "BFGS_LIMITS_TEST"
):
    """
    Systematic testing of BFGS with different noise levels

    Args:
        noise_levels: List of sigma values to test
        n_runs_per_noise: Number of independent runs per noise level
        output_dir: Directory to save results
    """

    print("="*80)
    print("BFGS_LIMITS_TESTING - SYSTEMATIC NOISE ANALYSIS")
    print("="*80)
    print(f"\nTest Configuration:")
    print(f"  Noise levels: {noise_levels}")
    print(f"  Runs per noise level: {n_runs_per_noise}")
    print(f"  Total runs: {len(noise_levels)*n_runs_per_noise}")
    print(f"  Output directory: {output_dir}")

    # Create output directory
    os.makedirs(output_dir, exist_ok=True)

    # Problem setup
    truss = Two_Bar_Plane_Truss_Problem()

    def objective_clean(x):
        sum_val, _, _ = truss.evaluate(x)
        return sum_val

    x0 = np.array([1.0, 1.5])
    lb = np.array([0.1, 0.5])
    ub = np.array([2.25, 2.5])
```

```
# CURRENT BFGS parameters (from your main_TBT_BFGS_noise.py)
opts = BFGSOptions(
    max_iter=1000,
    tol_grad=1e-5,
    tol_step=1e-12,
    fd_eps=1e-6,
    c1=1e-4,
    backtrack_beta=0.5,
    min_step=1e-12,
    verbose=False
)

print(f"\nBFGS_Parameters_(CURRENT/FIXED):")
print(f"max_iter={opts.max_iter}")
print(f"tol_grad={opts.tol_grad:.2e}")
print(f"tol_step={opts.tol_step:.2e}")
print(f"fd_eps={opts.fd_eps:.2e}")
print(f"c1={opts.c1:.2e}")
print(f"backtrack_beta={opts.backtrack_beta}")
print(f"min_step={opts.min_step:.2e}")

# Store all results
all_results = []

# Run tests
total_runs = len(noise_levels) * n_runs_per_noise
current_run = 0

print(f"\n{' '*80}")
print("STARTING_TESTS")
print(f"{' '*80}\n")

for sigma in noise_levels:
    print(f"\n{' '*80}")
    print(f"NOISE_LEVEL:  $\sigma$ ={sigma:.2e}")
    print(f"{' '*80}")

    for run in range(n_runs_per_noise):
        current_run += 1

        # Create noisy objective with different seed for each run
        seed = 42 + run # Reproducible but different per run
        objective_noisy = make_noisy_objective(objective_clean, sigma, seed=seed)

        # Run optimization
        print(f"Run_{run+1}/{n_runs_per_noise} (Global: {current_run}/{total_runs})...
              ", end=" ")
```

```
        result_dict = evaluate_bfgs_performance(
            objective_clean=objective_clean,
            objective_noisy=objective_noisy,
            x0=x0,
            lb=lb,
            ub=ub,
            opts=opts,
            sigma=sigma,
            problem_instance=truss,
            run_id=run + 1
        )

    all_results.append(result_dict)

    # Quick summary
    status = "✓" if result_dict['converged'] else "×"
    feasible = "✓" if result_dict['feasible'] else "×"
    print(f"{status} Conv | {feasible} Feas | {result_dict['iterations']:3d} iter | "
          "
          f"f={result_dict['f_final_clean']:.4e}")

# Convert to DataFrame (excluding history arrays)
results_summary = []
for r in all_results:
    r_copy = r.copy()
    # Remove history arrays for CSV
    r_copy.pop('history_f', None)
    r_copy.pop('history_grad', None)
    r_copy.pop('history_alpha', None)
    results_summary.append(r_copy)

df = pd.DataFrame(results_summary)

# Save raw results
csv_path = os.path.join(output_dir, "bfgs_limits_results.csv")
df.to_csv(csv_path, index=False)
print(f"\n✓ Results saved to: {csv_path}")

return df, all_results

# ***** ANALYSIS FUNCTIONS *****

def analyze_results(df: pd.DataFrame, output_dir: str):
    """
    Analyze and visualize results
    """

    print(f"\n{'='*80}")
```

```
print("STATISTICAL_ANALYSIS")
print(f"{' '*80}\n")

# Group by noise level
grouped = df.groupby('sigma')

# Summary statistics
summary = grouped.agg({
    'converged': ['sum', 'mean'],
    'feasible': ['sum', 'mean'],
    'iterations': ['mean', 'std', 'min', 'max'],
    'time_sec': ['mean', 'std'],
    'f_final_clean': ['mean', 'std', 'min', 'max'],
    'improvement_pct': ['mean', 'std'],
    'oscillation': ['mean', 'std'],
    'grad_norm_final': ['mean', 'std'],
}).round(6)

print("Summary_Statistics_by_Noise_Level:")
print(summary)

# Save summary
summary_path = os.path.join(output_dir, "summary_statistics.csv")
summary.to_csv(summary_path)
print(f"\n✓Summary saved to: {summary_path}")

# Identify breaking point
print(f"\n{' '*80}")
print("BREAKING_POINT_ANALYSIS")
print(f"{' '*80}\n")

for sigma in df['sigma'].unique():
    subset = df[df['sigma'] == sigma]
    conv_rate = subset['converged'].mean() * 100
    feas_rate = subset['feasible'].mean() * 100

    print(f"σ={sigma:.2e}:")
    print(f"Convergence_rate: {conv_rate:.1f}%")
    print(f"Feasibility_rate: {feas_rate:.1f}%")

    if conv_rate < 50:
        print(f"WARNING: Less than 50% convergence rate!")
    if feas_rate < 80:
        print(f"WARNING: Less than 80% feasible solutions!")
    print()

def plot_results(df: pd.DataFrame, all_results: List[Dict], output_dir: str):
    """
```

```
##### Create comprehensive visualization
#####

print(f"\n{'='*80}")
print("GENERATING VISUALIZATIONS")
print(f"{'='*80}\n")

noise_levels = sorted(df['sigma'].unique())

# ===== PLOT 1: Success Rates =====
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Convergence rate
conv_rates = [df[df['sigma'] == s]['converged'].mean() * 100 for s in noise_levels]
axes[0].plot(noise_levels, conv_rates, 'o-', linewidth=2, markersize=8, color='blue')
axes[0].axhline(y=50, color='red', linestyle='--', label='50% threshold')
axes[0].set_xlabel('Noise Level ( $\sigma$ )', fontsize=12, fontweight='bold')
axes[0].set_ylabel('Convergence Rate (%)', fontsize=12, fontweight='bold')
axes[0].set_title('BFGS Convergence vs Noise', fontsize=14, fontweight='bold')
axes[0].set_xscale('log')
axes[0].grid(True, alpha=0.3)
axes[0].legend()

# Feasibility rate
feas_rates = [df[df['sigma'] == s]['feasible'].mean() * 100 for s in noise_levels]
axes[1].plot(noise_levels, feas_rates, 'o-', linewidth=2, markersize=8, color='green')
axes[1].axhline(y=80, color='orange', linestyle='--', label='80% threshold')
axes[1].set_xlabel('Noise Level ( $\sigma$ )', fontsize=12, fontweight='bold')
axes[1].set_ylabel('Feasibility Rate (%)', fontsize=12, fontweight='bold')
axes[1].set_title('Solution Feasibility vs Noise', fontsize=14, fontweight='bold')
axes[1].set_xscale('log')
axes[1].grid(True, alpha=0.3)
axes[1].legend()

# Average iterations
avg_iters = [df[df['sigma'] == s]['iterations'].mean() for s in noise_levels]
axes[2].plot(noise_levels, avg_iters, 'o-', linewidth=2, markersize=8, color='purple')
axes[2].set_xlabel('Noise Level ( $\sigma$ )', fontsize=12, fontweight='bold')
axes[2].set_ylabel('Average Iterations', fontsize=12, fontweight='bold')
axes[2].set_title('Convergence Speed vs Noise', fontsize=14, fontweight='bold')
axes[2].set_xscale('log')
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(os.path.join(output_dir, '1_success_rates.png'), dpi=300, bbox_inches='tight')
print(f"✓ Saved: 1_success_rates.png")
plt.close()
```

```
# ===== PLOT 2: Convergence Metrics =====
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Final objective value
for sigma in noise_levels:
    subset = df[df['sigma'] == sigma]
    axes[0, 0].scatter([sigma]*len(subset), subset['f_final_clean'],
                      alpha=0.6, s=50, label=f' $\sigma={sigma:.0e}$ ')
axes[0, 0].set_xlabel('Noise Level ( $\sigma$ )', fontsize=11, fontweight='bold')
axes[0, 0].set_ylabel('Final Objective (Clean)', fontsize=11, fontweight='bold')
axes[0, 0].set_title('Solution Quality vs Noise', fontsize=12, fontweight='bold')
axes[0, 0].set_xscale('log')
axes[0, 0].set_yscale('log')
axes[0, 0].grid(True, alpha=0.3)
axes[0, 0].legend(fontsize=8)

# Final gradient norm
for sigma in noise_levels:
    subset = df[df['sigma'] == sigma]
    axes[0, 1].scatter([sigma]*len(subset), subset['grad_norm_final'],
                      alpha=0.6, s=50, label=f' $\sigma={sigma:.0e}$ ')
axes[0, 1].axhline(y=1e-5, color='red', linestyle='--', label='tol_grad=1e-5')
axes[0, 1].set_xlabel('Noise Level ( $\sigma$ )', fontsize=11, fontweight='bold')
axes[0, 1].set_ylabel('Final Gradient Norm', fontsize=11, fontweight='bold')
axes[0, 1].set_title('Gradient Convergence vs Noise', fontsize=12, fontweight='bold')
axes[0, 1].set_xscale('log')
axes[0, 1].set_yscale('log')
axes[0, 1].grid(True, alpha=0.3)
axes[0, 1].legend(fontsize=8)

# Oscillation
for sigma in noise_levels:
    subset = df[df['sigma'] == sigma]
    axes[1, 0].scatter([sigma]*len(subset), subset['oscillation'],
                      alpha=0.6, s=50, label=f' $\sigma={sigma:.0e}$ ')
axes[1, 0].set_xlabel('Noise Level ( $\sigma$ )', fontsize=11, fontweight='bold')
axes[1, 0].set_ylabel('Oscillation (Std of last 20%)', fontsize=11, fontweight='bold')
axes[1, 0].set_title('Solution Stability vs Noise', fontsize=12, fontweight='bold')
axes[1, 0].set_xscale('log')
axes[1, 0].set_yscale('log')
axes[1, 0].grid(True, alpha=0.3)
axes[1, 0].legend(fontsize=8)

# Improvement percentage
for sigma in noise_levels:
    subset = df[df['sigma'] == sigma]
    axes[1, 1].scatter([sigma]*len(subset), subset['improvement_pct'],
                      alpha=0.6, s=50, label=f' $\sigma={sigma:.0e}$ ')
axes[1, 1].set_xlabel('Noise Level ( $\sigma$ )', fontsize=11, fontweight='bold')
```

```
axes[1, 1].set_ylabel('Improvement_␣(%)', fontsize=11, fontweight='bold')
axes[1, 1].set_title('Optimization_␣Progress_␣vs_␣Noise', fontsize=12, fontweight='bold')
axes[1, 1].set_xscale('log')
axes[1, 1].grid(True, alpha=0.3)
axes[1, 1].legend(fontsize=8)

plt.tight_layout()
plt.savefig(os.path.join(output_dir, '2_convergence_metrics.png'), dpi=300, bbox_inches=
    'tight')
print("␣✓␣Saved:␣2_convergence_metrics.png")
plt.close()

# ===== PLOT 3: Convergence Curves =====
fig, axes = plt.subplots(2, 3, figsize=(18, 10))

for idx, sigma in enumerate(noise_levels[:6]): # Max 6 plots
    row = idx // 3
    col = idx % 3

    # Get all runs for this noise level
    runs_data = [r for r in all_results if r['sigma'] == sigma]

    # Plot each run
    for run_data in runs_data:
        axes[row, col].semilogy(run_data['history_f'], alpha=0.5, linewidth=1)

    axes[row, col].set_xlabel('Iteration', fontsize=10)
    axes[row, col].set_ylabel('Objective_␣Value', fontsize=10)
    axes[row, col].set_title(f'σ_␣={sigma:.2e}', fontsize=11, fontweight='bold')
    axes[row, col].grid(True, alpha=0.3)

plt.suptitle('Convergence_␣Curves_␣for_␣Different_␣Noise_␣Levels',
    fontsize=14, fontweight='bold', y=1.00)
plt.tight_layout()
plt.savefig(os.path.join(output_dir, '3_convergence_curves.png'), dpi=300, bbox_inches='
    tight')
print("␣✓␣Saved:␣3_convergence_curves.png")
plt.close()

# ===== PLOT 4: Boxplots =====
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Objective value boxplot
data_obj = [df[df['sigma'] == s]['f_final_clean'].values for s in noise_levels]
bp1 = axes[0, 0].boxplot(data_obj, labels=[f'{s:.0e}' for s in noise_levels])
axes[0, 0].set_xlabel('Noise_␣Level_␣(σ)', fontsize=11, fontweight='bold')
axes[0, 0].set_ylabel('Final_␣Objective_␣(Clean)', fontsize=11, fontweight='bold')
axes[0, 0].set_title('Objective_␣Value_␣Distribution', fontsize=12, fontweight='bold')
axes[0, 0].set_yscale('log')
```

```
axes[0, 0].grid(True, alpha=0.3, axis='y')

# Iterations boxplot
data_iter = [df[df['sigma'] == s]['iterations'].values for s in noise_levels]
bp2 = axes[0, 1].boxplot(data_iter, labels=[f'{s:.0e}' for s in noise_levels])
axes[0, 1].set_xlabel('Noise_Level_(\sigma)', fontsize=11, fontweight='bold')
axes[0, 1].set_ylabel('Iterations', fontsize=11, fontweight='bold')
axes[0, 1].set_title('Iteration_Count_Distribution', fontsize=12, fontweight='bold')
axes[0, 1].grid(True, alpha=0.3, axis='y')

# Gradient norm boxplot
data_grad = [df[df['sigma'] == s]['grad_norm_final'].values for s in noise_levels]
bp3 = axes[1, 0].boxplot(data_grad, labels=[f'{s:.0e}' for s in noise_levels])
axes[1, 0].axhline(y=1e-5, color='red', linestyle='--', label='tol_grad')
axes[1, 0].set_xlabel('Noise_Level_(\sigma)', fontsize=11, fontweight='bold')
axes[1, 0].set_ylabel('Final_Gradient_Norm', fontsize=11, fontweight='bold')
axes[1, 0].set_title('Gradient_Norm_Distribution', fontsize=12, fontweight='bold')
axes[1, 0].set_yscale('log')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3, axis='y')

# Time boxplot
data_time = [df[df['sigma'] == s]['time_sec'].values for s in noise_levels]
bp4 = axes[1, 1].boxplot(data_time, labels=[f'{s:.0e}' for s in noise_levels])
axes[1, 1].set_xlabel('Noise_Level_(\sigma)', fontsize=11, fontweight='bold')
axes[1, 1].set_ylabel('Time_(seconds)', fontsize=11, fontweight='bold')
axes[1, 1].set_title('Computation_Time_Distribution', fontsize=12, fontweight='bold')
axes[1, 1].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.savefig(os.path.join(output_dir, '4_distributions.png'), dpi=300, bbox_inches='tight')
print("\u2713 Saved: 4_distributions.png")
plt.close()

print("\n\u2713 All visualizations generated!")

# ===== MAIN EXECUTION =====

if __name__ == "__main__":

    # Define noise levels to test (logarithmic scale)
    noise_levels = [
        0.0,      # No noise (baseline)
        1e-6,     # Tiny noise
        1e-5,     # Very small
        1e-4,     # Small
        5e-4,     # Medium-small
```

```
1e-3,      # Medium
5e-3,      # Medium-large
1e-2,      # Large
5e-2,      # Very large
1e-1,      # Huge
5e-1,      # Extreme
1.0,      # Maximum tested
]

# Run tests
df, all_results = test_bfgs_limits(
    noise_levels=noise_levels,
    n_runs_per_noise=15, # 15 runs per noise level for statistical significance
    output_dir="BFGS_LIMITS_TEST"
)

# Analyze results
analyze_results(df, "BFGS_LIMITS_TEST")

# Plot results
plot_results(df, all_results, "BFGS_LIMITS_TEST")

print(f"\n{'='*80}")
print("TESTING_COMPLETE!")
print(f"{'='*80}")
print(f"\nAll results saved in: {BFGS_LIMITS_TEST}/")
print(f"{__bfgs_limits_results.csv__}(raw data)")
print(f"{__summary_statistics.csv__}(statistics)")
print(f"{__1_success_rates.png__}")
print(f"{__2_convergence_metrics.png__}")
print(f"{__3_convergence_curves.png__}")
print(f"{__4_distributions.png__}")
```

B Detailed Results Data

B.1 Sensor Calibration problem

B.1.1 Protocol A: Measurement Noise Summary (First 20 runs)

Seed	Solver	Status	NFEV	Final MSE	Time (s)
0	BFGS_Single	True	38	0.2428	0.0349
0	BFGS_Multi	True	36	0.2428	0.0108
1	BFGS_Single	True	38	0.2104	0.0090
1	BFGS_Multi	True	38	0.2104	0.0100
2	BFGS_Single	True	38	0.2231	0.0090
2	BFGS_Multi	True	38	0.2231	0.0098
3	BFGS_Single	True	38	0.2015	0.0091
3	BFGS_Multi	True	38	0.2015	0.0095
4	BFGS_Single	True	38	0.1876	0.0089
4	BFGS_Multi	True	38	0.1876	0.0102
5	BFGS_Single	True	38	0.2354	0.0092
5	BFGS_Multi	True	38	0.2354	0.0096
6	BFGS_Single	True	38	0.2198	0.0091
6	BFGS_Multi	True	38	0.2198	0.0099
7	BFGS_Single	True	38	0.2089	0.0089
7	BFGS_Multi	True	38	0.2089	0.0101
8	BFGS_Single	True	38	0.2312	0.0090
8	BFGS_Multi	True	38	0.2312	0.0094
9	BFGS_Single	True	38	0.2275	0.0093
9	BFGS_Multi	True	38	0.2275	0.0099

Table 16: Protocol A Data Sample (Measurement Noise)

B.1.2 Protocol B: Function Noise Summary (First 20 runs)

Seed	Solver	Status	NFEV	Final MSE	Time (s)
0	BFGS_Single	True	15	0.9263	0.0111
0	BFGS_Multi	True	18	0.0453	0.0094
1	BFGS_Single	True	18	1.8421	0.0093
1	BFGS_Multi	True	21	0.5123	0.0105
2	BFGS_Single	True	14	3.2109	0.0089
2	BFGS_Multi	True	19	1.1023	0.0101
3	BFGS_Single	True	12	5.6781	0.0085
3	BFGS_Multi	True	25	0.8912	0.0102
4	BFGS_Single	False	10	10.231	0.0080
4	BFGS_Multi	True	30	0.2312	0.0125

Table 17: Protocol B Data Sample (Function Noise)

B.1.3 Protocol C: Robustness Summary

Sigma	BFGS Single	BFGS Multi	SA	Hybrid
0.01	9.50e-05	9.50e-05	1.39e-04	9.50e-05
0.05	2.14e-03	2.14e-03	2.24e-03	2.14e-03
0.10	8.55e-03	8.55e-03	8.58e-03	8.55e-03
0.20	3.42e-02	3.42e-02	3.45e-02	3.42e-02
0.50	2.14e-01	2.14e-01	2.14e-01	2.14e-01
1.00	8.55e-01	8.55e-01	8.56e-01	8.55e-01
1.50	1.92e+00	1.92e+00	1.93e+00	1.92e+00
2.00	3.42e+00	3.42e+00	3.43e+00	3.42e+00
3.00	7.70e+00	7.70e+00	7.71e+00	7.70e+00
5.00	2.14e+01	2.14e+01	2.14e+01	2.14e+01

Table 18: Protocol C Robustness Results

B.2 Two Bar Plane Truss problem

Test	Convergence	Iterations	Evaluations	Final gradient norm	Final step size
1	False	30	121	1.05e-06	1.16e-10
2	False	30	121	1.05e-06	1.16e-10
3	False	30	121	1.05e-06	1.16e-10
4	False	30	121	1.05e-06	1.16e-10
5	True	24	97	1.20e-06	1.00e+00
6	True	24	97	1.20e-06	1.00e+00
7	False	30	121	1.05e-06	1.16e-10
8	False	30	121	1.05e-06	1.16e-10
9	False	24	97	1.66e-04	1.00e+00
10	True	24	97	1.20e-06	1.00e+00
11	True	24	97	1.20e-06	1.00e+00
12	True	30	121	1.22e-03	1.00e-12
13	True	24	97	1.20e-06	1.00e+00
14	True	24	97	1.19e-06	1.00e+00
15	True	24	97	1.34e-06	1.00e+00
16	True	13	53	2.31e-06	1.00e+00
17	True	15	61	1.06e-07	1.00e+00
18	False	50	201	9.26e-05	1.00e+00
19	True	39	157	2.42e-08	1.00e+00

Table 19: Algorithm results

Test	x1 [norm]	x2 [norm]	Pos [m]	Area [m^2]	Weight [kg]	Disp [m]	Ten [Pa]	Comp [Pa]
1-11	0.657	0.533	1.67	3.44e-4	16.37	8.67e-3	5.64e4	-1.09e8
12	0.657	0.533	1.67	3.44e-4	16.37	8.67e-3	5.64e4	-1.09e8
16	0.657	0.513	1.67	3.31e-4	15.77	8.99e-3	5.24e6	-1.08e8
17	0.657	0.531	1.67	3.42e-4	16.31	8.70e-3	5.60e5	-1.09e8
18	0.657	0.533	1.67	3.44e-4	16.37	8.67e-3	5.65e3	-1.09e8
19	0.657	0.533	1.67	3.44e-4	16.37	8.67e-3	5.65e2	-1.09e8

Table 20: Optimisation results

C Group Evaluation

Group Member	Mark	Commentary
Pierre BEDRUNE	5	All assigned tasks have been completed. GENERAL: Explore a lot of ideas for each task. TASK 1&2: Create the Gantt on Notion. TASK 3: Code and write half of the Two Bar Plane Truss problem; Code Simulated Annealing and run the simulation; Write explanations and discussions. TASK 4: Code Noise exploration and simulation; Write explanations and discussions.
Martin HARRAMBAT	5	All assigned tasks have been completed. TASK 1: Analyse his problem and understand it. TASK 2: Show concern in the deep understanding of the project. TASK 3: Code the full Gear Train Design, cost function; Wrote explanations and discussed SA vs PSO. TASK 4: Change the fundamentals of the gear train to fit BFGS; Wrote explanations and explained findings.
Clémence-Philomène HINOT	5	All assigned tasks have been completed. GENERAL: Update Gantt on Notion. Create base code for Two-Bar Truss. TASK 1&2: Find Two bar truss problem and analyse it. TASK 3: Code PSO algorithm; Wrote half of discussion on Two-Bar Truss; Updated Gantt; Code Two bar truss problem. TASK 4: Code base BFGS; Code parametrisation optimization; Wrote part on code optimization problem.
John HOARAU	5	All assigned tasks have been completed. GENERAL: Manage the group and organise meetings; Bring together all work; Ensure everyone has information. TASK 1&2: Organise tasks in Notion. TASK 3: Code and write Sensor Calibration problem; Code Simulated Annealing; Adapt PSO; Write explanations. TASK 4: Adapt Sensor Calibration for BFGS; Run simulation with/without noise; Write explanations and discussions.

Table 21: Group Evaluation